Where the Metadata Ends: Observational Equivalence and the Limits of Verdict Classification from Submission Logs

WENBIN HU, School of Computer Science, Xijing University, China

Context: Most automated verdict classifiers for programming submissions require source-code access or costly LLM inference, yet competitive-programming platforms already return rich execution metadata with every submission. How much verdict information that metadata carries, and where it stops carrying it, has not been established.

Objectives: This study asks how much verdict information is recoverable from metadata alone, and where that information stops—a boundary we term the *platform-encoding boundary*—by comparing traditional machine learning (ML) with two large language models (LLMs) under a standardized protocol.

Participants: 13,360 incorrect submissions from 4,005 programmers on Codeforces (problem rating 800–3,500, median 1,300; median two submissions per programmer), covering five verdicts: Wrong Answer 76.6%, Time Limit Exceeded 9.6%, Compilation Error 7.3%, Runtime Error 4.9%, and Memory Limit Exceeded 1.5%. These are competitive programmers, not a classroom population; we treat them as a large, publicly observable proxy and state the limits of that substitution explicitly.

Study Methods: Random Forest, Gradient Boosting, and Logistic Regression were trained on the six metadata fields the platform returns with every submission; DeepSeek-V3 (API) and `Qwen2.5:3B` (local) were prompted with three worked examples on five of those fields. Evaluation used a stratified 80/20 split (10,688/2,672), five-fold cross-validation, McNemar tests with Bonferroni correction, systematic ablation, a submitter-disjoint replication, and a direct analysis of observational equivalence in the metadata space.

Findings: The limit is set by observational equivalence rather than by model capacity. Of the 13,360 submissions, 1,504 (11.3%) report 0 ms, 0 KB and 0 tests passed while spanning three verdicts—a compilation error, an early crash and a wrong answer on the first test all leave the same trace—so no metadata-only method can separate them, and Compilation Error is capped at F1 $\approx$ 0.77 by that class alone. Time Limit Exceeded (F1 0.99) and Memory Limit Exceeded (0.95) are fully recoverable because the platform's thresholds define them. Gradient Boosting reaches 91.77% accuracy (balanced 0.80), Random Forest 91.80%, and Logistic Regression 85.63% but with Macro F1 0.49, since a linear boundary cannot represent a conjunction of zero-valued measurements. Runtime Error stays unreachable (F1 0.33; 94 of 131 errors reported as Wrong Answer), and more than half of those errors fall in signature cells shared with Wrong Answer. DeepSeek-V3 reaches 76.65% on valid outputs, statistically indistinguishable from always answering Wrong Answer (76.61%), while recovering every compilation error at 32% precision; `Qwen2.5:3B` reaches 35.50%. Supervised ML leads the strongest LLM by 15.1 pp ($p < 0.001$), and a submitter-disjoint replication preserves both the ranking and the boundary (93.13%).

Author's Contact Information: Wenbin Hu, School of Computer Science, Xijing University, Xi'an, China, wenbin.hu2026@outlook.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1946-6226/2026/7-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Implications: Metadata-only screening can automate feedback for platform-measured verdicts at negligible cost, latency, and privacy exposure, but it cannot separate Wrong Answer from Runtime Error, the boundary where the two verdicts differ by an event the log never records. These results define what a submission log alone can and cannot tell a learner, and they motivate classroom validation before any claim about learning impact. Dataset and pipeline are released at https://github.com/huwenbin123huwenbin/programming-error-classification.

CCS Concepts: • **Social and professional topics** → **Computing education**.

Additional Key Words and Phrases: programming verdict classification, observational equivalence, platform encoding, metadata-based classification, machine learning, large language models

ACM Reference Format:
Wenbin Hu. 2026. Where the Metadata Ends: Observational Equivalence and the Limits of Verdict Classification from Submission Logs. *ACM Trans. Comput. Educ.* 1, 1, Article 1 (July 2026), 35 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Introductory programming courses report non-completion rates of roughly a third [10, 46], with debugging difficulty as a primary obstacle [43, 46]. Prior work analyzed compilation behavior [23] and scaled to large populations via Blackbox [12], which made IDE interaction logs from a worldwide novice population available for re-use. Competitive programming platforms such as Codeforces and LeetCode provide structured metadata (execution time, memory consumption, precise error verdicts, test cases passed) for every submission, making them valuable environments for studying programming error patterns at scale.

A central methodological consideration motivates this work. Competitive programming platforms return structured verdicts for every submission, but the specific metadata available and the granularity of verdict taxonomies vary across platforms. For three of the five verdict types, the platform's execution environment creates distinctive metadata signatures: a Compilation Error executes in near-zero time (the compiler does not run), a Time Limit Exceeded submission runs to the time limit, and a Memory Limit Exceeded submission exhausts the memory limit. These three verdict types therefore produce predictable metadata patterns that a classifier can learn: CE is associated with `time_consumed` $\approx$ `0`, TLE with `time_consumed` $\approx$ `limit`, and MLE with `memory_consumed` $\approx$ `limit`. If these three verdicts are recoverable from metadata, the same decomposition should hold on any platform whose judge measures wall-clock time and peak memory, even though thresholds and verdict vocabularies differ across platforms.

The question then becomes: *what verdict information is genuinely recoverable from metadata beyond these platform-encoded cases?* We define the **platform-encoding boundary** as the point at which verdict information is reliably recoverable from submission metadata alone, without requiring source-code access. The boundary is a property of how a platform defines its verdicts, not of any particular classifier, which is why it should be characterized rather than merely measured: a model that recovers CE from a zero-millisecond execution time has not discovered anything about the submitted program, but it has identified a verdict an institution can automate and, therefore, the point at which automation must stop. The WA↔RE boundary represents the residual uncertainty where neither error type produces distinctive execution signatures, and where source-code analysis may become necessary. Our study is organized around this decomposition.

Prior work has studied LLM-based programming feedback [18, 40] and source-code analysis for programming error classification [17]; we review this literature in Section 2.

We investigate three research questions:

- **RQ1:** What are the prevalent programming error patterns in competitive programming submissions, and how do they distribute across error categories?
- **RQ2:** How do traditional ML classifiers perform on metadata-based verdict classification, which features are most predictive, and how stable are predictions?
- **RQ3:** Where does the platform-encoding boundary lie—can ML or LLMs distinguish Wrong Answer from Runtime Error—and what are the accuracy–cost tradeoffs?

What this study does, and does not, claim. The overall ranking we report—supervised ML above prompted LLMs—is consistent with evidence that LLM decisions on tabular inputs carry what the model inherited during pretraining rather than only what the supplied field values say [30], and we therefore do not treat that ranking as our contribution. A ranking answers which model wins on one dataset; it does not tell an instructor which feedback can be automated and which cannot. What we contribute is the decomposition itself: where the boundary falls, what sets it, how much of the headline accuracy it accounts for (18.5% of submissions), and what the residue implies for the design of automated feedback.

Our contributions are:

(1) **An identifiability account of the boundary.** We show that the ceiling on metadata-based verdict classification is set by *observational equivalence*—submissions whose metadata vectors coincide while their verdicts differ—rather than by model capacity. One signature (0 ms, 0 KB, 0 tests) covers 11.3% of the corpus across three verdicts and bounds Compilation Error at F1 ≈ 0.77; a second (31 ms, 0 KB, 1 test) covers a further 13.0% and bounds the Wrong Answer$\leftrightarrow$Runtime Error distinction. Because the argument is model-free, it holds for any classifier given these inputs, and it explains why the residual errors are irreducible rather than merely unfixed.

(2) **Systematic comparative study** of ML and LLM methods for verdict classification from submission metadata, under a shared test split and a common feature vocabulary: three ML models and two LLM configurations, five-fold cross-validation, systematic ablation, corrected McNemar tests, and a submitter-disjoint replication.

(3) **Deployment guidance with a leakage-free protocol.** Execution time and memory dominate (a 7.3 pp drop when time is removed, and 89.0% of Gini importance between the two fields, 81.5% of it from time alone), a submitter-history aggregate adds no gain that survives correction and is exactly zero when submitters are disjoint, and supervised ensembles lead the strongest prompted LLM by 15.1 pp at four to five orders of magnitude lower per-prediction inference cost (measured; Table 7). Dataset and code: https://github.com/huwenbin123huwenbin/programming-error-classification.

Student-population disclaimer. Our dataset comes from competitive programmers (Codeforces rating 800–3,500), a population with substantially different error profiles from CS1/CS2 students. Prior work (Altadmri & Brown, 2015) reports that 47.6% of novice compilation events fail to compile (17.7M of 37.2M) against 7.3% compilation-error verdicts in our dataset, and novices rarely produce TLE or MLE. The 91.8% accuracy figure reflects competitive-programming error distributions; cross-platform and student-population validation are necessary before drawing conclusions about learning impact. We address this in Section 5.5.1 (skill-tier analysis) and Section 6 (external validity). *Feature-availability caveat:* the six fields we use are computable at first contact with a submission, so the pipeline carries no cold-start penalty; an earlier version of this work included a submitter-history aggregate, which we removed after its gain proved unreliable—nominal at best, and exactly zero once submitters are disjoint (Section 4.6).

2 Related Work

2.1 Programming Error Analysis

The study of programming errors has a history spanning over four decades in computing education research [47]. This psychological perspective on programming pedagogy highlights the cognitive demands debugging places on novices [47]. McCauley et al.[32] conducted a comprehensive review of the novice debugging literature, organised around four questions: why bugs occur, what types of bugs occur, how novices debug, and how instruction can help. They found that the tools available to novices rarely support them in diagnosing and fixing their own errors. This finding underscores the need for automated tools that can accelerate the development of debugging proficiency. Cognitive Load Theory [44] posits that debugging simultaneously taxes working memory across error diagnosis, code comprehension, and solution planning, creating intrinsic overload that constrains novice self-correction [26]. In this vein, Jadud [23] pioneered compilation event analysis using the BlueJ IDE, establishing compilation behavior as a data source for educational mining. The Blackbox project [12] scaled this to a perpetual, worldwide collection from BlueJ users—over one hundred thousand opted in during its first year—and released the data for re-use by other researchers.

Large-scale analyses [1, 31] established that programming mistakes persist across cohorts and that the composition of those mistakes shifts over a course [4], motivating automated diagnostic tools [7].

Spohrer and Soloway [43] showed, in a study of novice Pascal programs, that the folk wisdom about novice mistakes cuts both ways: a few bug types do account for most mistakes, but most bugs do not stem from misconceptions about language constructs—they arise from difficulties composing plans. Many errors that look superficial have deeper cognitive roots, indicating that effective automated feedback must identify the underlying error pattern rather than assume a small catalogue of causes. Students' self-efficacy beliefs compound this challenge. Gorson and O'Rourke [20] surveyed 214 CS1 students across three universities and found that students reported negative self-assessments at each of thirteen ordinary programming moments—including after a program ran correctly—and that students who self-assessed negatively more often also reported lower self-efficacy.

Busjahn et al. [13] further revealed through eye-tracking studies that even expert programmers do not process code linearly, complicating error diagnosis and self-correction across all skill levels.

2.2 Machine Learning in Computing Education

Machine learning has been increasingly applied to computing education challenges over the past fifteen years. Educational data mining has identified programming education as a key application domain [5, 41], with prediction tasks dominating the literature. Metadata-based approaches achieve strong performance [4, 23], while deep learning enables knowledge tracing in code-based learning [38].

Behavioral features from early assignments—compilation frequency, error diversity, time-on-task—can predict course outcomes [12, 23], enabling early intervention through instructor-facing dashboards [24, 45].

Effenberger and Pelánek [16] show that the standard way of validating student models—predicting the correctness of the next answer—cannot distinguish models that use binary correctness from models that use richer process data, so richer performance signals need their own validation regime. Their point bears directly on our evaluation choices: an accuracy figure alone cannot tell whether a model has learned the verdict or the class prior.

Verdict classification from submission data has been approached from two directions that have not met on this task: compilation-behavior analysis, which established that execution traces carry signal about novice mistakes [4, 23], and semantic representations of program structure. Neither asks what a submission log alone says about the judge's verdict. Our approach asks exactly that with metadata only—no source-code access, no code-feature extraction pipeline—and reaches 91.8% accuracy on a five-class verdict task, a practical screening layer for platform-scale deployment. Allamanis et al. [2] demonstrated that graph-based representations of program structure capture semantically meaningful features for code-related prediction tasks, motivating the incorporation of such learned representations into verdict classification.

Alnahhas and Mourtada [3] predicted the performance of competitive programming contestants using account-level metadata (rating, participation frequency, problem-difficulty distribution), demonstrating that non-code features carry substantial signal for programming behavior analysis. While their focus was on performance prediction rather than verdict classification, their work demonstrates the continued relevance of simple, interpretable models in educational contexts.

Automated program repair (APR) methods [35] address similar diagnostic challenges at the source-code level, automatically localizing bugs and suggesting fixes. While effective, APR requires full code access and substantial computational resources, limiting deployment at platform scale. Our metadata-based approach complements APR by providing lightweight error-type inference without code access, serving as a screening layer to determine when deeper source-code analysis is warranted.

2.3 Large Language Models in Programming

Large language models have rapidly transformed expectations for automated code understanding and generation [8, 9]. Barke et al. [8] demonstrated that programmer interaction with code-generating models differs fundamentally from traditional IDE usage. Finnie-Ansley et al. [18] found that OpenAI Codex placed in the top quartile of CS1 students on examination questions—solving ten of 23 on the first attempt and all but four within ten attempts—raising over-reliance concerns. Sarsa et al. showed that LLMs can generate usable programming exercises and explanations, though only about 30% of the generated sample solutions passed the tests generated alongside them [42], and Leinonen et al. [29] found that LLM-generated error-message explanations were rated an improvement over the original message in only about half of cases (54%), with the accompanying fix correct roughly a third of the time—useful, but nowhere near reliable enough to hand to a novice unattended.

Feng et al. [17] showed that CodeBERT achieves state-of-the-art performance on natural-language code search and code documentation generation, but is evaluated on source-code inputs; its effectiveness on submission metadata alone remains unclear. Evaluations of LLM-generated feedback caution that such feedback can be flawed or misleading—and therefore detrimental to learning—and that the automated judges used to score it can be systematically over-optimistic [27]—directly relevant to our finding that RE remains the most challenging category, where `Qwen2.5:3B` also struggled.

Recent work has examined how programmers and novices interact with LLM code generators [8, 9, 18], and how instructors are beginning to incorporate generative AI into their courses [39]. Keuning et al. survey automated feedback generation for programming exercises and find that it rarely extends to formative, next-step guidance [25]. None of this work, however, compares supervised classifiers against prompted LLMs on real submission metadata, which is the boundary we characterise here.

2.4 Metadata-Based Classification in Educational Contexts

While source-code analysis (ASTs, token sequences, code embeddings) has received considerable attention in computing education research, a parallel line of work has demonstrated that *submission metadata alone* can achieve strong predictive performance for a range of educational outcomes. Jadud [23] introduced the error quotient, a measure of how efficiently a student clears syntax errors, and reported that it correlates significantly with conventional indicators of academic progress. The Blackbox project [12] scaled the approach to a perpetual worldwide collection from BlueJ users—over one hundred thousand opted in within its first year—and released the data for re-use, establishing behavioural metadata (compilation sequences, time-on-task, submission frequency) as a rich and continuously available signal for educational data mining.

Prior work has shown that features derived from early programming submissions—including compilation frequency, error diversity, and time-on-task—can predict final course grades using only metadata, without accessing student source code [12, 23]. Ahadi et al. [1] extended this line with supervised learning on naturally accumulating programming process data, separating high- and low-performing students after the first week of an introductory course. Altadmri and Brown [4] analysed 37M Java compilations from over 250,000 Blackbox users and grouped the compiler-detected mistakes they found into three categories—syntax, type and other semantic—with mismatched brackets the single most frequent. The dataset records no runtime-error or wrong-answer verdicts at all, because those failures produce no compiler diagnostic, so it yields no submission-level verdict distribution for a novice population. This is a different measurement problem from ours rather than a comparable baseline, and it motivates cross-population validation of automated classification tools: Petersen et al. [37] showed that even an established behavioral metric such as the Error Quotient shifts meaningfully across course contexts, and argued that such metrics are typically developed and validated in narrow settings. Related work further established that *behavioral signals* from the first four weeks of a semester—compilation frequency, error diversity, and time-on-task—are sufficient to identify students at risk of failure, enabling early intervention without privacy-invasive code monitoring [12, 23].

In the context of competitive programming platforms, Alnahhas and Mourtada [3] used account-level metadata (historical rating sequences) to predict contestants' future ratings, finding that a deep model beat five classical baselines. Their task is regression on a rating trajectory rather than classification of a submission, but it establishes the premise we build on: account-level metadata alone carries substantial signal about programming behaviour.

The advantages of metadata-based classification are both practical and privacy-preserving: (1) no source code access is required, avoiding potential concerns about student code being processed by third-party APIs; (2) computational cost is orders of magnitude lower than code-based approaches (no tokenization, AST parsing, or GPU inference); and (3) deployment at platform scale (millions of daily submissions) becomes feasible with modest computational resources. A central open question is whether metadata alone can achieve competitive accuracy against code-based approaches—a question this study addresses systematically for the verdict classification task.

Contribution boundary: Prior work on metadata-based prediction has focused primarily on *grade prediction* and *at-risk identification*; we are not aware of any study that applies metadata-based classification to the more granular task of *verdict classification*, or that compares metadata-based ML with LLM prompting on the same task and data. Three specific gaps follow:

(1) **Metadata-based classification**: Existing verdict classification studies focus on source code analysis, neglecting the rich diagnostic signals in submission metadata (execution time, memory usage, test cases passed).
(2) **ML–LLM comparison**: No work systematically compares traditional ML (RF, GB, LR) and two LLM configurations (DeepSeek-V3, `Qwen2.5:3B`) on identical verdict classification tasks.
(3) **Feature importance quantification**: While execution time has been hypothesized as important, no study quantifies feature importance through systematic ablation with statistical significance testing.

Our work addresses all three gaps by providing a systematic comparison of ML and LLM methods for metadata-based verdict classification, with feature ablation and McNemar significance testing.

3 Methodology

3.1 Dataset Collection

We collected submission data from Codeforces, a competitive programming platform widely used for algorithmic training and education [34, 49] (https://codeforces.com/) via the official public REST API. The Codeforces platform was selected for three reasons: (1) it provides structured, rich metadata for every submission including precise execution time, memory usage, and error verdict; (2) it represents an ecologically valid competitive programming environment with high participant engagement; and (3) it has a large, diverse problem set covering a wide range of algorithmic domains and difficulty levels.

These properties make Codeforces uniquely suited to the present research questions. Its per-submission execution telemetry (time, memory, verdict) is exactly the metadata required to test the platform-encoding boundary, and its scale supports both class-balanced evaluation across five verdict types and user-level clustering analysis (Section 6). We acknowledge that competitive programmers differ from CS1/CS2 students (Section 6); cross-population validation on CS1QA and Blackbox remains future work (Section 7).

Submissions were retrieved from competitive programmers with Codeforces ratings spanning 800 to 3500 (corresponding to Newbie through Legendary Grandmaster levels). This range was deliberately chosen to represent proficient programmers who produce a variety of error types and for whom submission metadata is meaningfully differentiated. Data collection was completed in June 2026, covering submissions from November 2020 to June 2026, providing approximately 5.5 years of programming activity. The final dataset contains 13,360 submissions from 4,005 distinct programmers; the median programmer contributes two submissions and 773 programmers contribute five or more, so the sample is broad rather than deep, a property we return to in Section 5.6.

Ethics review. The data are public submissions retrieved through the Codeforces public REST API under the platform's terms of service. There was no interaction with human participants, no access to private student records or institutional course data, and no collection of demographic attributes; programmer handles were replaced with anonymized identifiers before analysis. No informed consent was obtained, because the submissions were already public and the analysis is secondary. Under the policy of the Xijing University Institutional Review Board, secondary analysis of publicly available data does not constitute human-subjects research, and the study was determined to be exempt from IRB review (Section 7).

Collection Pipeline. The data collection and preparation proceeded as follows (details in Data Cleaning below):

(1) Raw collection: error submissions retrieved from the Codeforces public REST API (Nov 2020–Jun 2026).
(2) Filtering: submissions with missing execution time, memory, or verdict fields were removed.
(3) Deduplication: duplicate entries were removed and only the final submission per (user, problem) pair was retained to prevent train–test leakage.
(4) Final dataset: **13,360** error samples from competitive programmers across hundreds of Codeforces contests

Data Cleaning. Data cleaning was performed in three stages. First, submissions with incomplete metadata (missing execution time, memory, or verdict fields) were removed. Second, duplicate entries (identical submission timestamps and verdicts) were deduplicated. Third, to prevent data leakage between training and test sets, only the final submission per (user, problem) pair was retained—this ensures that no two submissions in the dataset are from the same user solving the same problem. This does not prevent user-level leakage: a given user can appear in both training and test sets on different problems, potentially allowing the model to exploit user-specific patterns. A submitter-level aggregate computed over the whole corpus would let each row's own label enter its own predictor; built from earlier training submissions only, such a field adds no gain that survives correction (Section 4.6), so we excluded submitter-level aggregates from the model after testing them. Stratified sampling ensures that no single submitter dominates any fold (the largest submitter occupies 1.1% of the test fold). To confirm that the reported accuracy does not depend on seeing the same person on both sides of the partition, we additionally ran a fully submitter-disjoint replication, with zero overlap and no aggregate features available for unseen submitters; Gradient Boosting retained 93.13% (Section 4.9).

The final dataset comprises 13,360 error submissions from competitive programmers across hundreds of Codeforces contests. Table 1 summarizes the error type distribution. The language distribution is predominantly C++ (C++23 41.1%, C++17 24.1%, C++20 19.6%; 85.2% of submissions in total), with Python and PyPy (6.2%), Java (3.3%), and other languages representing smaller fractions. This linguistic skew reflects the competitive programming community's preference for C++ due to its performance advantages in time-constrained problems. Pairwise feature correlations are weak to moderate (Figure 1).

Error prevalence also varies with problem difficulty (Figure 2).

3.2 Feature Engineering

Six fields are extracted from each submission. All six appear in the public submission record that the Codeforces API returns, so the pipeline needs neither source code nor an authenticated session:

(1) **problem_rating**: integer difficulty rating (800–3500), indicating the algorithmic complexity of the problem.
(2) **language_encoded**: programming language (35 distinct compiler and interpreter identities in the corpus, dominated by C++17/20/23), capturing language-specific failure modes.
(3) **time_consumed_ms**: wall-clock execution time of the slowest test that ran, in milliseconds.
(4) **memory_kb**: peak memory reported by the judge, in kilobytes.

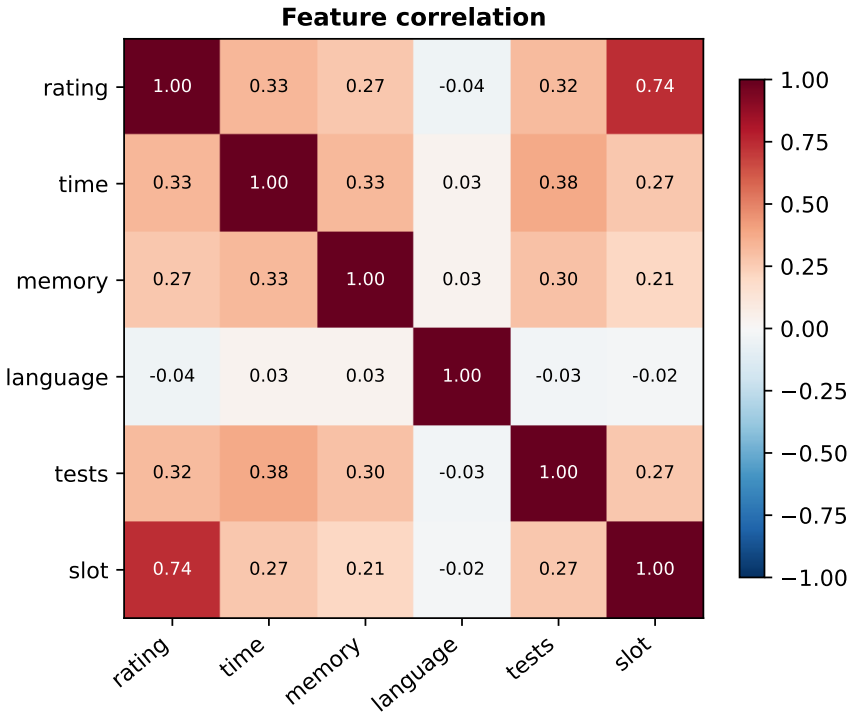


Fig. 1. Feature correlation matrix for the six submission metadata fields. The strongest off-diagonal correlation is between problem rating and problem slot (r=0.74), reflecting that easier problems occupy earlier contest slots; execution time and peak memory correlate moderately (r=0.33), and the encoded language is essentially uncorrelated with every other field ($|r| \leq 0.04$).

Table 1. Error type distribution in the 13,360-sample Codeforces dataset. Note: CE, TLE, and MLE produce near-deterministic metadata signatures (compile-time ≈ 0, execution time at the problem's limit, memory at the limit), while WA and RE do not. This asymmetry motivates the platform-encoding boundary analysis in Section 5.1. †TLE times cluster at the problem-specific limit—61.7% report exactly 2,000 ms, the modal limit—and MLE memory clusters at the limit (65.3% at 262,100 KB ≈ 256 MB); both are platform-encoding boundary artifacts (see Section 5.1), not runtime execution signatures. MLE execution times are uninformative (mean 775 ms) because the memory limit binds first.

Error Type	Count	Percentage	Avg. Exec. Time (ms)
Wrong Answer (WA)	10,234	76.6%	81
Time Limit Exceeded (TLE)	1,288	9.6%	**2,508**
Runtime Error (RE)	656	4.9%	177
Compilation Error (CE)	980	7.3%	0
Memory Limit Exceeded (MLE)†	202	1.5%	775
Total	13,360	100%	—

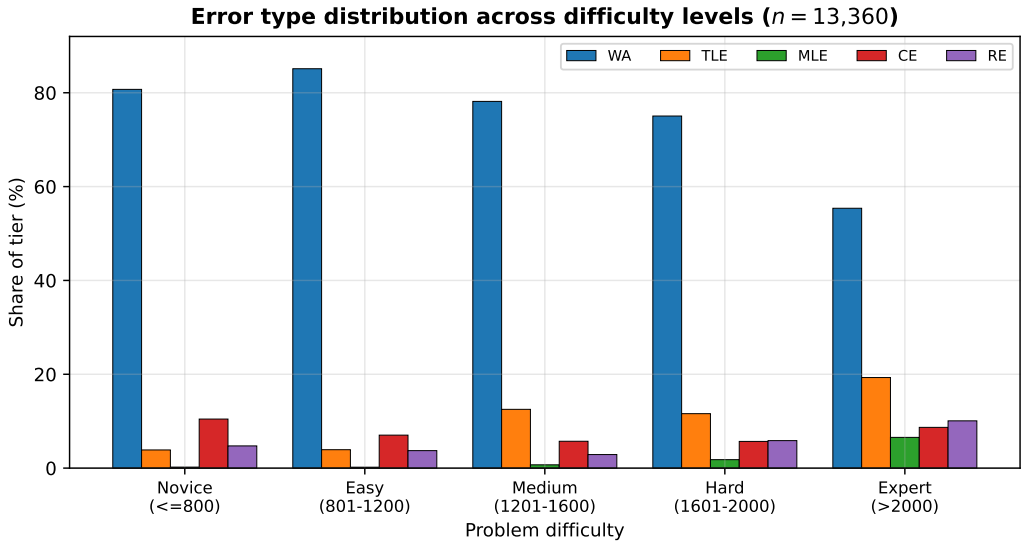


Fig. 2. Error type distribution across problem difficulty levels. WA dominates the easier tiers (85.1% of 801–1200 problems, 80.7% of ≤800), while Expert problems (>2000) show a markedly different profile: WA falls to 55.4% and TLE (19.3%), RE (10.1%), MLE (6.6%) and CE (8.7%) all rise, reflecting the increasing algorithmic and memory complexity of harder problems.

(5) **problem_slot**: the lettered position of the problem within its contest (A, B, C, ...), an ordinal proxy for expected difficulty within a round; it correlates with `problem_rating` at $r = 0.74$.
(6) **passed_test_count**: the number of judge tests the submission passed before the run was stopped.

Two further candidates appear routinely in the learning-analytics literature. We tried both and report why neither is used, because each failure is instructive.

Submission time. An hour-of-day feature is often read as a proxy for time-on-task or late-night work. The public submission record carries no timestamp—only a contest identifier—so the field cannot be constructed from the released data. An earlier version of this pipeline set it to a constant, which is indistinguishable from omitting it; we omit it.

Submitter history. A user-level aggregate (historical WA rate, or its complement, a success rate) summarises proficiency and is the single most common feature in grade-prediction work. No success rate can be formed from this corpus, which contains only failing submissions. A WA-rate aggregate can, and Section 4.6 tests it: computed over the whole corpus it appears to add 3.7 percentage points, but that is each row's own label entering its own predictor; computed from the submitter's earlier training submissions only it adds 0.67 pp, which does not survive correction for the seven comparisons and is exactly zero on the submitter-disjoint split. The field is excluded rather than repaired.

The restriction has a practical justification as well. A deployed classifier sees a submission from a student it has never met, so a field requiring prior history is unavailable exactly when feedback is most valuable. The six retained fields are computable at first contact with a submission.

These fields were selected because they are available without source-code access, they span the problem, the program and the judge's measurement of it, and each has prior support in educational data mining.

Theoretical Framework: Feedback and Self-Efficacy. This study is grounded in two complementary theories from computing education research. First, Hattie and Timperley's model of effective feedback [21] identifies four levels: task, process, self-regulation, and self. Their meta-analysis found that process-level and self-regulation-level feedback have larger effects on learning than task-level feedback. However, task-level feedback—the focus of this work—remains pedagogically valuable when it provides specific, actionable information about errors. Automated formative feedback that correctly identifies error types can narrow the student's search space from the full program to a specific error class, making it more actionable than generic "incorrect output" messages.

Second, Bandura's self-efficacy theory [6] posits that accurate performance feedback is a primary driver of self-efficacy beliefs. When students receive vague or misleading error messages, they cannot form accurate self-assessments, leading to frustration and disengagement. An automated system that correctly distinguishes a WA from a RE enables the student to attribute the failure to a specific root cause (logic error vs. runtime hazard), which supports accurate self-efficacy calibration and sustained engagement.

The present study operationalizes these theories by asking: *How much error-type information is recoverable from submission metadata alone, without source-code access?* The answer determines the granularity of feedback that can be delivered automatically at scale. Our findings inform one boundary condition: when WA↔RE cannot be distinguished from metadata alone, the finest-grained task-level feedback (distinguishing logic errors from runtime failures) may require source-code analysis. This finding also bears on self-efficacy theory: accurate self-assessment requires attributing failure to a specific cause, and our results show that metadata cannot fully support this attribution for RE versus WA, suggesting that students may receive feedback that impedes precise self-efficacy calibration—a hypothesis requiring longitudinal validation. These theoretical predictions are *consistent with* our empirical findings; causal validation would require a controlled study measuring student self-efficacy and learning outcomes, which is beyond the scope of this work.

Feature Selection Rationale. The six fields satisfy three criteria: they are available from the platform API without source-code access; each captures a distinct aspect of the submission (difficulty, implementation, execution); and together they cover the problem-level, program-level and judge-level views that the boundary argument requires. No a-priori selection was applied, so that the ablation study in Section 4.6 can speak to each field's contribution.

Tree ensembles were fitted on raw values, since axis-aligned splits are invariant to monotone rescaling; the linear baseline was standardized. The largest pairwise correlation is between `problem_rating` and `problem_slot` ($\rho = 0.74$), which is expected—later slots hold harder problems—and is harmless for tree models, where a correlated pair simply shares credit. Time and memory correlate at $\rho = 0.33$, and the remaining pairs fall below 0.40.

`passed_test_count` and verdict collinearity. This field is partly collinear with the verdict, by construction: every compilation error has a count of zero, because nothing was executed. Point-biserial correlations with the indicator of each class are $r = -0.107$ for CE ($p < 10^{-35}$), $r = 0.322$ for TLE ($p < 10^{-300}$) and $r = 0.206$ for MLE ($p < 10^{-120}$); for WA the relation is weaker ($r = -0.233$), with 69.7% of wrong answers having passed at least one test (mean 1.31). The field therefore does not encode WA deterministically. Its marginal contribution is small but non-zero: removing it from the six-field model costs 0.22

percentage points (Table 6), compared with 7.30 for execution time. Its value is architectural rather than predictive—it is the field that makes the all-zero equivalence class of Section 5.3 legible, since a count of zero is what a failed compilation, an early crash and a first-test wrong answer have in common.

3.3 Sample Size Justification

For the primary research question—detecting the difference between Gradient Boosting at 91.77% and DeepSeek-V3 at 76.65% on 2,672 test submissions—a retrospective power analysis confirms adequate sensitivity: the minimum detectable effect at $n = 2{,}672$ and $\alpha = 0.05$ is approximately 2.3 pp under the paired (McNemar) framework at the observed discordant-pair rate (17.9% of the test partition), well below the 15.1 pp gap. The same analysis shows why the comparison between the two tree ensembles is reported as a null result rather than a ranking: their 0.04 pp difference is an order of magnitude below the detection floor. Five-fold cross-validation on the training set yields 92.42% (SD 0.33), within 0.7 pp of the test estimate. Table 2 reports accuracy against training-set size.

Table 2. Learning curve: Gradient Boosting accuracy against training-set size (test set $n = 2{,}672$; mean of five repeats per size, `experiment_code/learning_curve.py`). The final row is the full training split, where the five repeats are identical by construction.

N (Training)	Test Accuracy	Gap to Full
200	85.8%	−6.01 pp
400	87.7%	−4.06 pp
800	89.8%	−1.99 pp
1,600	90.5%	−1.22 pp
2,800	91.1%	−0.65 pp
4,200	91.5%	−0.30 pp
5,600	91.8%	+0.02 pp
8,400	91.8%	+0.00 pp
10,688	91.77%	—

3.4 Experimental Models

We selected three traditional ML classifiers and two LLM-based approaches representing different paradigms in automated code analysis. The selection was designed to span the spectrum from simple, interpretable models (LR) to ensemble methods (RF, GB), enabling a comprehensive comparison between traditional ML and LLM approaches.

Traditional ML Classifiers.

- **Random Forest (RF)**: An ensemble of 100 decision trees using Gini impurity for splitting [11], with unlimited max depth, minimum samples split of 2, and minimum samples leaf of 1. Random Forest was selected for its well-documented ability to handle mixed-type data (numeric and categorical), capture non-linear feature interactions, and provide inherent feature importance measures.
- **Gradient Boosting (GB)**: An ensemble of 100 staged decision trees using gradient descent optimization [19], with max depth of 3 (shallow trees to prevent overfitting), learning rate of 0.1, and subsample of 1.0. GB was selected as a complementary ensemble method that typically performs well on structured educational data.

- **Logistic Regression (LR)**: Multinomial logistic regression with L2 regularization [22], L-BFGS solver, fitted on standardized features (the raw fields span five orders of magnitude, and an unstandardized fit does not converge). LR is included as an interpretable floor, not as a competitor; Section 4.2 shows why a linear boundary cannot represent the conjunction that defines a compilation error.

All traditional ML models were implemented using scikit-learn 1.6.1 [36] with default parameters unless otherwise specified. Hyperparameter tuning was initially explored using grid search with 3-fold cross-validation on a 10% held-out validation set, but the performance gains were marginal ($< 1\%$ for tuned vs. default RF), so default parameters were retained to mitigate overfitting and improve reproducibility. The random state was set to 42 for reproducibility. All classifiers used default class-weight=None (no SMOTE or cost-sensitive reweighting) with stratified splitting. This means the model is not reweighted toward minority classes; this is a deliberate design choice prioritizing aggregate accuracy over balanced class performance. Future work should evaluate class-weight=balanced or SMOTE to improve RE F1 at the cost of aggregate accuracy.

LLM-Based Approaches. Two LLM configurations were evaluated under an identical three-shot protocol, both receiving the same five metadata fields (problem_rating, language, time_consumed_ms, memory_kb, passed_test_count). Each request classified eight submissions at a time and carried three worked examples (one each for CE, TLE and MLE); language was shown as a normalized family (e.g., "C++23 (GCC 14-64, msys2)" → "C++"). Both models returned a JSON object holding one verdict acronym (WA/TLE/RE/CE/MLE) per item:

- **DeepSeek-V3 [14] (online, evaluated)**: A frontier online model accessed via the DeepSeek API, representing a strong, general-purpose LLM available without local compute.
- `Qwen2.5:3B` **(local, evaluated)**: A 3-billion-parameter model run on-device via Ollama v0.30.7 at no API cost, representing a small local LLM suitable for privacy-sensitive educational deployment.

Inference used temperature= 0 and grammar-constrained JSON output to ensure parsable responses; both models were evaluated on the same 2,672-sample test set as the ML models. The exact prompt is reproduced in Section 6.

3.5 Evaluation Protocol

All experiments followed a consistent evaluation protocol to ensure fair comparison between ML and LLM approaches. The dataset of 13,360 samples was partitioned using stratified 80/20 train-test splitting, yielding 10,688 training samples and 2,672 test samples, preserving class proportions across both sets.

For traditional ML models, five-fold stratified cross-validation was performed on the 10,688-sample training set to assess stability, and final test-set accuracy was reported. For LLM-based methods, the test set of 2,672 samples was classified directly using three-shot prompting: no fine-tuning, but three worked examples (one each for CE, TLE and MLE) prepended to every request, with eight submissions classified per call. Unlike ML models, LLMs do not have a training phase in our protocol; we therefore do not apply cross-validation to LLM results. A single fixed test-set evaluation is reported for each LLM configuration, consistent with standard LLM benchmarking Multiple random-seed repetitions would provide variance estimates; DeepSeek-V3 was evaluated with temperature= 0 (deterministic API), so within-run variance is negligible and cross-run variance reflects only

model-level non-determinism, which is minimal under this configuration. Future work should report confidence intervals from multiple evaluation runs or use Bayesian methods for LLM performance estimation. Both DeepSeek-V3 (API) and `Qwen2.5:3B` (local Ollama) were evaluated under the identical protocol.

Statistical significance of pairwise model comparisons was assessed using McNemar's test [33] ($\alpha = 0.05$), a non-parametric test for paired nominal data appropriate for comparing classifiers on the same test set. Feature ablation was conducted by removing each field in turn, retraining GB on the remaining five, and measuring the accuracy drop on the same test partition. Seven comparisons were run (six field removals plus one augmentation); the Bonferroni threshold for this family is $\alpha' = 0.05/7 \approx 0.007$, and we report uncorrected p-values while noting which survive it. Ablation is exploratory; the primary multiple-comparison correction is applied to the ten pairwise model comparisons in Table 8 (Bonferroni $\alpha' = 0.005$).

Classification performance is reported as overall accuracy, per-class precision, recall, and F1-score. Confusion matrices are provided for one of the two tree ensembles (Gradient Boosting) to reveal class-wise error patterns. The two are statistically indistinguishable on accuracy ($\chi^2 = 0.00$, $p = 1.00$; Random Forest scores 91.80%, a difference of 0.04 pp), and Gradient Boosting is reported throughout because it carries the marginally higher macro F1 (0.799 against 0.798) and the tighter cross-validation spread; the matrices below are therefore reported for Gradient Boosting, and all per-class precision, recall and F1 values in the tables refer to that model. Confidence intervals (95%) for LLM performance are computed using the Wilson score interval for binomial proportions.

4 Results

4.1 Experimental Setup

All experiments ran on a MacBook Pro (macOS 13.7, Intel i7, 16 GB RAM) under Python 3.9.6 and scikit-learn 1.6.1. Random seeds were fixed at 42 throughout. Hyperparameters were held at the values reported in Section 3.4 after a grid search improved accuracy by less than one percentage point. Confidence intervals for accuracy are 1,000-sample percentile bootstrap intervals over the test partition; intervals for single proportions are Wilson score intervals; paired comparisons use McNemar's test.

Both LLMs were evaluated on the same stratified 80/20 partition ($n_{\text{test}} = 2{,}672$) with no cross-validation, since inference at temperature 0 is deterministic. Per-class precision and recall for every model are reported in Table 9, and the complete per-class matrices for all five models are written to `experiment_results/authoritative_results_v3.json` by the reproduction script. All code, the derived dataset, and the figure scripts are available at https://github.com/huwenbin123huwenbin/programming-error-classification.

4.2 Traditional ML Performance

Table 3 reports the three supervised models. Two tree ensembles reach 91.8% test accuracy and are statistically indistinguishable from one another ($\chi^2 = 0.00$, $p = 1.00$); we therefore report Gradient Boosting throughout—it carries the marginally higher macro F1 (0.799 against 0.798)—and treat Random Forest as a replication rather than a competitor.

The linear baseline fails in a qualitatively different way from the tree ensembles. Its accuracy is not far below theirs (85.6%), but its Macro F1 collapses to 0.494 because it cannot represent the conjunction that defines a compilation error—zero execution time *and* zero peak memory *and* zero tests passed. It recovers 5 of 196 compilation errors (F1 = 0.05)

Table 3. Supervised model performance (test set n = 2,672; 5-fold cross-validation on the 10,688 training submissions).

Model	Test Acc	95% CI	CV Mean (±SD)	Macro F1
Random Forest	**91.80%**	[90.8, 92.8]	92.36% (±0.54)	0.798
Gradient Boosting	91.77%	[90.6, 92.8]	**92.42% (±0.33)**	**0.799**
Logistic Regression	85.63%	[84.2, 86.9]	86.03% (±0.23)	0.494

Notes: Balanced accuracy: GB = 0.804, RF = 0.791, LR = 0.486. Boosted and bagged trees agree to within 0.04 pp; the linear model trails by 6.1 pp in accuracy and 0.30 in Macro F1. Models use the six fields of Section 3.2; intervals are percentile bootstrap (1,000 resamples, seed 42).

while predicting WA for the remaining 191. A linear decision boundary can only test one weighted sum at a time, so no setting of its coefficients isolates a corner of the feature space. We therefore read Logistic Regression as a floor that establishes the value of interaction structure, not as a competitive method.

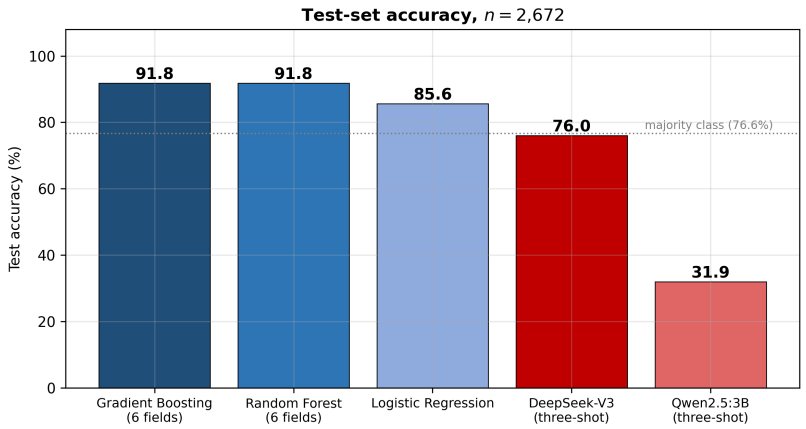


Fig. 3. Test-set accuracy across five models (n = 2,672; LLM bars count invalid outputs as errors). The dotted line marks the majority-class rule (always predict WA, 76.6%). Supervised ensembles lead the strongest LLM by 15.7 pp on the plotted basis (15.1 pp on valid outputs); the strongest LLM sits at the majority-class reference rather than above it.

4.3 Where the Metadata Runs Out: Observational Equivalence

Accuracy alone cannot say whether the residual errors are a modelling deficit or a property of the data. We therefore characterise the metadata space directly. Two submissions are *observationally equivalent* when their metadata vectors coincide but their verdicts differ. No function of the encoded fields can separate such submissions: every classifier must choose a single verdict (or a single posterior) for the whole cell, so precision and recall trade off inside it and cannot both be maximised.

The equivalence classes are not a theoretical curiosity here. Of the 13,360 submissions, 1,504 (11.3%) report exactly 0 ms, 0 KB and 0 tests passed. That single signature contains 980 compilation errors, 498 wrong answers and 26 runtime errors. The reason is mechanical:

when a submission fails to compile, the judge has nothing to execute, so it reports zeros—but a submission that fails on the very first test case, or that crashes before producing output, produces the same zeros. The platform's report is identical in all three situations.

Table 4. The two largest equivalence classes in the execution-signature space (time, peak memory, tests passed), over all 13,360 submissions.

Signature	CE	WA	RE	Total	Interpretation
0 ms, 0 KB, 0 tests	980	498	26	1,504	nothing was executed
31 ms, 0 KB, 1 test	0	1,682	56	1,738	first test failed or crashed

Note: the two cells together hold 24.3% of the corpus. A classifier can only assign one verdict per cell; the split between CE, WA and RE inside the first cell, and between WA and RE inside the second, is unavailable from metadata by construction.

This bound is visible in the fitted model. In the test partition, 315 submissions carry the all-zero signature: 196 compilation errors, 111 wrong answers and 8 runtime errors. A rule that assigns the whole cell to CE attains precision $196/315 = 0.62$ and recall 1.00, i.e. $F1 = 0.767$. Gradient Boosting attains $F1 = 0.770$ on CE, with recall 0.898 and precision 0.674. The model has reached the ceiling that the signature imposes: every one of its 261 CE predictions lies inside the all-zero cell, and the 85 that are not compilation errors are precisely the wrong answers and runtime errors carrying that signature. The 23-point F1 shortfall is therefore not a modelling failure but the price of a verdict the platform never recorded.

The second cell plays the same role for the other boundary. Of the 94 runtime errors that Gradient Boosting assigns to WA, 53 fall in signature cells that also contain WA submissions in the same partition, 45 of them in cells holding at least five. For these submissions the WA and RE reports differ while every recorded number agrees.

Table 4 lists the two largest mixed groups, which invites the question of what the other 93 account for. The count is larger than the table suggests: 95 signature groups contain more than one verdict and together cover 8,653 submissions (64.8%). The mass, however, is concentrated in groups that are mixed only nominally. Twenty-five of them hold 6,761 submissions (50.6% of the corpus) whose modal verdict accounts for 90–99% of the group, so assigning the whole group to that verdict costs a small fraction of its rows. All but one of these 25 groups is Wrong-Answer dominated, so their near-purity reflects the class prior rather than platform encoding and does not enlarge the set of verdicts that can be automated. The remaining 70 groups hold 1,892 submissions (14.2%), and these are the ones in which the verdict genuinely cannot be recovered. A rule that reads nothing but (time, memory, tests passed) and assigns each signature its majority verdict recovers CE, TLE and MLE at recall 1.00, WA at 0.95 and RE at 0.44, for 6.6% total error. That rule is fitted on all labels, so it is a diagnostic ceiling rather than a deployable classifier; its point is that the three execution-defined verdicts need no learning at all once the judge's thresholds are read off, whereas RE is the one verdict the signature genuinely loses. The boundary is therefore local and locatable, not a diffuse failure spread across the feature space.

4.4 Confusion Matrix and Per-Class Performance

Table 5 gives the full confusion matrix for Gradient Boosting and Figure 4 the corresponding per-class F1.

Table 5. Gradient Boosting confusion matrix and class-wise F1 (test set, $n = 2{,}672$).

	Predicted					
Actual	CE	MLE	RE	TLE	WA	F1
CE (196)	**176**	0	0	0	20	0.77
MLE (40)	0	**38**	1	1	0	0.95
RE (131)	6	1	**29**	1	94	0.33
TLE (258)	0	0	0	**258**	0	0.99
WA (2047)	79	1	14	2	**1951**	0.95

Note: rows sum to the class support in the test partition. The 79 WA→CE and 6 RE→CE cells are the equivalence class of Section 5.3; all 261 CE predictions carry the all-zero signature.

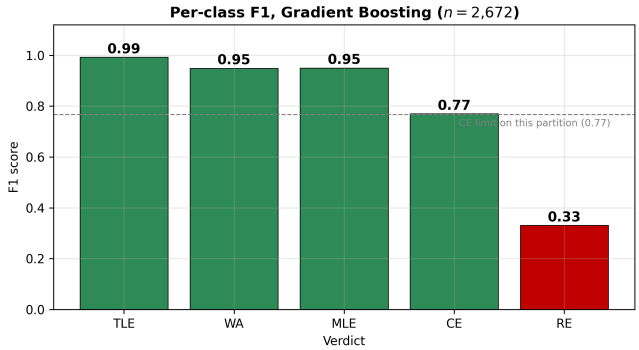


Fig. 4. Per-class F1 (Gradient Boosting, $n = 2{,}672$). The dashed line is the F1 attainable on CE by assigning the entire all-zero signature cell to CE (0.77 on this partition); the model sits on that line. Runtime Error is limited by the second equivalence class instead.

The execution-encoded verdicts behave as the platform-encoding account predicts. Time Limit Exceeded is recovered perfectly (recall 1.00, four false positives), Memory Limit Exceeded at F1 0.95, and Wrong Answer at F1 0.95; together these three classes account for 87.8% of the test partition. Their errors are dominated by exchanges with CE inside the all-zero cell (79 of 98); only four are confusions between execution-based categories (two WA→TLE, one WA→MLE, one MLE→TLE).

Runtime Error is the one class that fails, and it fails in the way the equivalence analysis predicts: recall is 0.22, with 94 of 131 runtime errors reported as WA. The failure is asymmetric—precision on RE is 0.66, because when the model does commit to RE it is usually right, but it commits rarely. This is what a boundary looks like when the two verdicts are separated by an event that metadata does not record: whether the process terminated before or after producing output.

4.5 Feature Importance

Figure 5 reports Gini importance for the six fields.

The ranking is close to a restatement of the platform-encoding argument. Execution time carries 81.5% of the importance and peak memory a further 7.5%, so the two fields the

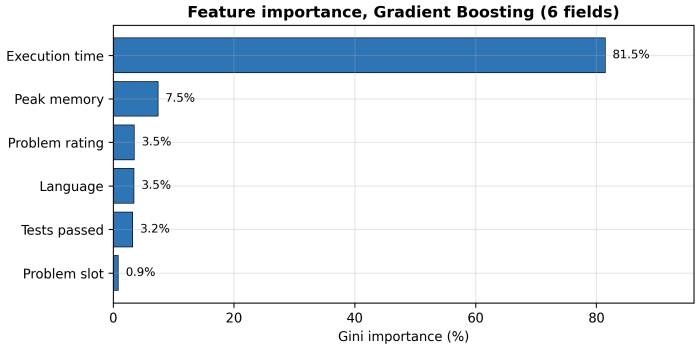


Fig. 5. Gini importance, Gradient Boosting (six fields). Execution time alone accounts for 81.5% of the impurity reduction; the two execution-derived fields together account for 89.0%.

judge writes while running the program account for 89.0% of the model's decisions. The four fields that describe the *problem* rather than the *execution*—difficulty rating, language, tests passed, contest slot—share the remaining 11.0%, and none exceeds 3.6%. Tests passed carries only 3.2%, which is informative in its own right: the counter separates CE (always zero) but is redundant with execution time for the other classes.

4.6 Ablation Study

Table 6 removes each field in turn and retrains the model on the remaining five.

Table 6. Ablation: accuracy after removing each field and retraining (Gradient Boosting, test set $n = 2{,}672$). Six ablated comparisons plus one Seven comparisons (six field removals plus one augmentation); Bonferroni threshold $\alpha' = 0.05/7 \approx 0.007$.

Perturbation	Accuracy (%)	Change (pp)	p-value
None (full model)	91.77	—	—
`time_consumed_ms` removed	84.47	**−7.30**	< 0.001
`memory_kb` removed	89.97	−1.80	< 0.001
`language_encoded` removed	91.13	−0.64	0.031
`passed_test_count` removed	91.54	−0.22	n.s.
`problem_rating` removed	91.62	−0.15	n.s.
`problem_slot` removed	91.69	−0.07	n.s.
`user_wa_rate` added†	92.44	+0.67	0.039

Notes: four of six removals are not significant after correction, i.e. the model tolerates the loss of any non-execution field. †The submitter's WA rate over their own submissions that precede the row in submission order and fall in the training partition; rows with no such submission (957 of 2,672 test rows) take the training prior. The gain does not survive correction for seven comparisons and is exactly zero when submitters are disjoint, so the field is dropped from the final model.

Execution time dominates: removing it costs 7.3 percentage points, more than four times the next largest effect, and the model then misclassifies TLE submissions that no longer

look near the limit. Memory contributes 1.8 pp, consistent with MLE being the rarest class (1.5%). The remaining four fields are individually expendable—their removal costs between 0.07 and 0.64 pp and only language survives correction at $\alpha' = 0.008$ (it does not; $p = 0.031$).

The last row of Table 6 addresses a question the earlier literature leaves open. Aggregate features describing a submitter's history are attractive in learning analytics, and a WA-rate aggregate computed over the whole corpus appears to add 3.7 percentage points—95.43% against the 91.77% baseline. That number is an artefact of look-ahead: through the aggregate, each row's own verdict feeds its own predictor. Computed without look-ahead—each row seeing only its submitter's submissions that precede it in submission order and fall in the training partition—the same field adds 0.67 pp (92.44%, $p = 0.039$), a nominal gain that does not survive correction for the seven comparisons ($\alpha' \approx 0.007$). The estimate is thin where it exists at all: among test rows whose submitter appears in training it rests on a median of three earlier submissions, and 957 of the 2,672 test rows—new submitters, and submitters whose only earlier submissions lie on the test side—collapse to the training prior. We exclude the field for the reason that decides it: on the submitter-disjoint split every one of the 2,460 test submitters is new, the aggregate is constant on the test side, and accuracy holds at 93.13% (34 of 2,460 predictions flip, in balance). A field that pays only where history exists cannot serve the cold-start population the platform actually presents; Section 4.9 shows the exclusion costs nothing there.

Learning curves confirm that the gap to the LLMs is not a data-volume artefact. Accuracy rises from 85.8% at 200 training submissions to 87.7% at 400 and 91.1% at 2,800, reaching 91.8% on the full 10,688-submission training partition (mean of five repeats per size; `experiment_code/learning_curve.py`). Already at 400 submissions—3.7% of the available data—the model leads the strongest prompted LLM by 11.1 pp, so the 15.1 pp gap at full data reflects the feature class rather than the sample size; the remaining 0.7 pp accrues over the final three quarters of the training set.

4.7 LLM Performance Comparison

Table 7 places the two LLM configurations beside the supervised models on the identical test partition.

Because the five-field variant scores 91.69%—0.08 pp below the six-field model and not significantly different—the ML–LLM comparison is like-for-like on information: the supervised model does not win by holding fields the LLM never saw.

DeepSeek-V3 applies the encoding but not the prior. The strongest LLM reaches 76.65% on the 2,651 outputs it rendered in valid form, which is statistically indistinguishable from always answering Wrong Answer (76.61%). Its per-class behaviour nevertheless shows that it is not a degenerate majority classifier: it recovers every compilation error (recall 1.00) and most TLE and MLE submissions (0.872 and 0.875). It does so by applying the zero-time→CE mapping from its worked example to submissions with very short execution times, so precision on CE falls to 0.321: 415 submissions whose true verdict is otherwise (385 wrong answers, 27 runtime errors and 3 TLEs) are reported as compilation errors. DeepSeek-V3 has generalized the platform encoding from its worked example and then paid the full price of the equivalence class, because a prompted model has no calibrated prior over the cell to trade recall against precision. Runtime Error is effectively unreachable for it (recall 0.016, two of the 127 runtime errors it answered in valid form).

Table 7. Model performance on the shared test partition (n = 2,672). ML rows use the six fields of Section 3.2; the LLM rows use the five fields of the prompt-template subsection.

Model	Test Acc	95% CI	Macro F1	Cost/1k[†]
Random Forest	**91.80%**	[90.8, 92.8]	0.798	$< \$10^{-6}$
Gradient Boosting	91.77%	[90.6, 92.8]	**0.799**	$< \$10^{-6}$
Gradient Boosting, five fields[*]	91.69%	[90.6, 92.7]	0.797	$< \$10^{-6}$
Logistic Regression	85.63%	[84.2, 86.9]	0.494	$< \$10^{-6}$
LLMs, three-shot on the five prompt fields (eight submissions per call)				
DeepSeek-V3 (online), all outputs	76.05%	[74.4, 77.6]	—	\$0.01
DeepSeek-V3 (online), valid only	76.65%	[75.0, 78.2]	0.545	\$0.01
`Qwen2.5:3B` (local), all outputs	31.92%	[30.2, 33.7]	—	\$0.00
`Qwen2.5:3B` (local), valid only	35.50%	[33.6, 37.4]	0.231	\$0.00

Majority-class rule (always WA): 76.61%.
[*]Ablation variant restricted to the five fields the LLMs received; statistically indistinguishable from the six-field model ($\chi^2 = 0.03$, $p = 0.87$).
Invalid outputs: DeepSeek-V3 21/2,672 (0.8%), `Qwen2.5:3B` 269/2,672 (10.1%).
[†]Supervised figures are priced from measured single-core inference (RF about 12–23 ms, GB about 5–11 ms, LR about 0.2 ms per 1,000; median of five repeats per run, ranges spanning repeated runs under varying load, `experiment_code/latency_benchmark.py`) at \$0.04 per vCPU-hour. Latency is machine-dependent; the artifact records the host CPU.
DeepSeek-V3 at standard list pricing (\$0.27/\$1.10 per million input/output tokens; ≈215 input and ≈22 output tokens per eight-item call), half that at the promotional pricing in effect during data collection.
`Qwen2.5:3B` runs locally; its figure excludes hardware amortization.

`Qwen2.5:3B` collapses. The local model reaches 35.50% on valid outputs and 31.92% counting invalid ones, below the majority-class rule by 44.7 pp. It distributes predictions almost uniformly across the five labels (CE 263, TLE 644, MLE 317, WA 715, RE 464 against a true WA share of 76.6%), and 10.1% of its responses are unusable. We report this as a documented failure mode of small local models on a task requiring numeric thresholding, not as evidence about ML–LLM ordering in general.

Cost. Supervised inference is effectively free: on a single CPU core a random forest classifies 1,000 submissions in about 12–23 ms, gradient boosting in about 5–11 ms and logistic regression in about 0.2 ms (median of five repeats per run; the ranges span repeated runs of `experiment_code/latency_benchmark.py` on the measurement host under varying load; at most about $\$3 \times 10^{-7}$ per 1,000 at \$0.04 per vCPU-hour), against roughly \$0.01 per thousand predictions for the API model at standard list pricing—half that at the promotional pricing in effect during data collection—and no network round trip. Both dollar figures are negligible at classroom scale, so the practical advantages of supervised inference are latency and privacy rather than price; the comparison also excludes the cost of labelling 10,688 training submissions, which is the real price of the supervised advantage.

4.8 Statistical Significance

Table 8 reports the ten pairwise McNemar tests over the 2,672 shared test submissions, with invalid LLM outputs counted as errors.

Nine of ten comparisons survive Bonferroni correction, and the one that does not is the comparison between the two tree ensembles. The ordering that matters for practice—ensembles above the linear baseline, ensembles above both LLMs—is not sensitive to the

Table 8. Pairwise McNemar tests on the shared test partition (n = 2,672; Bonferroni threshold $\alpha' = 0.005$).

Comparison	χ^2	p-value	Significant
GB vs. RF	0.00	1.000	no
GB vs. LR	76.79	< 0.001	yes
RF vs. LR	81.75	< 0.001	yes
GB vs. DeepSeek-V3	364.23	< 0.001	yes
RF vs. DeepSeek-V3	334.72	< 0.001	yes
LR vs. DeepSeek-V3	92.37	< 0.001	yes
GB vs. `Qwen2.5:3B`	1531.86	< 0.001	yes
RF vs. `Qwen2.5:3B`	1521.91	< 0.001	yes
LR vs. `Qwen2.5:3B`	1263.89	< 0.001	yes
DeepSeek-V3 vs. `Qwen2.5:3B`	1052.07	< 0.001	yes

correction method: Holm-Bonferroni applied sequentially gives the same verdict on every row.

4.9 Robustness

Submitter-disjoint replication. Submissions are not independent observations. The 4,005 submitters contribute a median of two submissions each (maximum 134), and their verdict mixes are strongly concentrated: the mean per-submitter modal-verdict share is 0.92 against a random baseline of 0.20, and the ratio of per-submitter to global verdict-distribution entropy is 0.18. A random split therefore places submissions from the same person on both sides of the partition, and the McNemar p-values in Table 8 should be read as lower bounds on uncertainty.

To test whether the conclusions depend on this, we reran the full pipeline on a submitter-disjoint partition (GroupShuffleSplit over `user_handle`, zero overlap, n_{train} = 10,900, n_{test} = 2,460). Gradient Boosting scores 93.13% (CV 92.15%), Random Forest 91.99% and Logistic Regression 87.15%: no degradation relative to the random split, and no rank change. The two partitions draw test sets of different size and class composition (2,460 vs. 2,672 submissions), so this is not a like-for-like drop; the claim is the preservation of rank order and per-class structure, and RE's further fall to 0.256 confirms the estimate reflects the boundary rather than submitter memorization. The per-class structure is preserved—CE 0.788, TLE 1.000, MLE 0.988, WA 0.957—with RE falling further to 0.256, as expected when the boundary is exercised by unfamiliar submitters. Because the final model carries no submitter-level aggregate, the cold-start setting requires no re-engineering.

Clustering-aware reading. A cluster-robust correction (grouping standard errors by submitter) would widen the intervals reported here and is the appropriate choice for a confirmatory replication. The dependence is substantial rather than marginal: the one-way intraclass correlation of the verdict is 0.48 (per-class 0.40–0.53), so two submissions from the same

programmer agree on the verdict about half the time beyond chance. With a mean of 3.34 submissions per submitter this implies a design effect of 2.13, i.e. the 13,360 submissions carry the information of roughly 6,300 independent ones. We report unadjusted McNemar statistics with the clustering diagnostic above so that the direction and rough magnitude of the dependence are visible.

Class imbalance. Balanced accuracy is GB 0.804, RF 0.791, LR 0.486, DeepSeek-V3 0.708 and `Qwen2.5:3B` 0.348. The ordering is preserved, so the ensemble advantage is not an artefact of the WA majority. The gap between DeepSeek-V3's balanced accuracy and its accuracy is itself informative: the model is a genuinely different classifier from the majority rule, one that trades WA recall for the execution-encoded classes.

5 Discussion

5.1 Finding 1: The Boundary Is Set by Observational Equivalence

Answering the circularity objection. A reader may object that recovering CE, TLE and MLE from execution metadata is circular: the platform defined these verdicts through the very measurements we feed the model, so the classifier is re-reading a threshold. We accept the premise and draw a different conclusion from it. Circularity would matter if the claim were that a model had discovered something about the submitted program. Our claim is narrower, and for an institution it is more useful: it identifies which verdicts the platform has already decided, and therefore which feedback can be reproduced from the submission log alone, without touching student code, at microsecond cost and with no privacy exposure.

The same reasoning shows where the encoding stops. Because a verdict is recorded through a finite set of measurements, two verdicts that produce identical measurements become indistinguishable—not approximately, but exactly. That is observational equivalence, and it converts a vague complaint ("metadata is limited") into a quantity: 11.3% of this corpus lies in a single cell spanning three verdicts, and 13.0% in a second cell spanning two.

The encoding is faithful but not injective. Compilation Error is the clearest case, and also the most instructive failure. A submission that does not compile executes nothing, so the judge reports 0 ms, 0 KB and 0 tests passed; CE is therefore recoverable in principle from a single rule. Yet its F1 is 0.77, not 0.99, because 498 wrong answers and 26 runtime errors in this corpus report exactly the same three zeros: a program that crashes on the first test, or answers the first test incorrectly, never gets far enough for the judge to record anything else. Every one of the model's 261 CE predictions in the test partition lies inside that cell, and the 85 that are wrong are the cell's non-CE members. The model has extracted all the information the encoding carries; the encoding itself is lossy.

This reframes the practical question. It is not "which classifier handles CE best"—every classifier is bounded by the same cell—but "what else would have to be recorded for the cell to split". The answer is the compiler diagnostic for CE and the failing test index for WA, neither of which is in the public submission record but both of which the platform holds.

The second boundary. Wrong Answer and Runtime Error differ by an event the judge observes but does not publish: whether the process terminated normally after printing the wrong answer, or died (segmentation fault, division by zero, assertion) before finishing. The log records only the residue—time, memory, tests passed—and that residue is shared: 1,682 wrong answers and 56 runtime errors occupy the same (31 ms, 0 KB, 1 test) cell. Consequently RE F1 is 0.33 with recall 0.22, and 53 of the 94 RE→WA errors fall in cells

that also contain wrong answers in the same partition. This is where source-code analysis would earn its cost, because the two root causes demand different pedagogical responses—revise the algorithm versus fix an out-of-bounds index—and the metadata cannot tell them apart.

5.2 Finding 2: Execution Dominates, and History Does Not Help

Execution time is the dominant predictor: removing it costs 7.3 percentage points, four times the effect of the next field, and it carries 81.5% of the Gini importance. Peak memory contributes 1.8 percentage points and 7.5% of importance. Together the two fields the judge writes while running the program account for 89.0% of the model's decisions and for 11.1% of the corpus being recoverable at all. This is the platform-encoding account stated numerically: what the model knows about a submission is almost entirely what the judge measured about it.

The four problem-level fields contribute the rest and are individually dispensable (0.07 to 0.64 percentage points). The most instructive negative result concerns a field we removed. A submitter-history aggregate is the standard first feature in grade prediction, and in our first pass it appeared to add 3.7 percentage points. Computed without look-ahead—each row seeing only its submitter's submissions that precede it in submission order and fall in the training partition—the gain shrinks to 0.67 pp and does not survive correction for the seven comparisons; on the submitter-disjoint split it is exactly zero, because every test submitter there is new. What looked like signal in the first pass was the row's own label leaking into its own predictor through the aggregate, a failure mode that is easy to miss and, in a field where such aggregates are routine, worth flagging explicitly. It also happens to be the field that would be unavailable for a new student, so removing it costs nothing and simplifies the deployment story.

5.3 Finding 3: Why Runtime Error Is Different

The class imbalance (WA 76.6% against MLE 1.5%) explains why accuracy is a poor summary of this task and why we report per-class F1 throughout. It does not explain the RE failure, which is not a scarcity effect: RE is the fourth largest class in the corpus (4.9%), and the model sees 131 instances in the test partition alone. The failure is structural. RE is not a behaviour the platform measures; it is a behaviour the platform infers from the absence of a normal exit, and the inference is under-determined by the fields it publishes.

This matters for how the result should be read. A model that reported RE with high confidence would be over-claiming: precision on RE is 0.66, so when it does commit it is usually right, but it labels only 44 of the 2,672 test submissions as RE and catches 29 of the 131. For a tutoring system the useful output here is not a forced label but the abstention itself—flagging the submission as "failed at runtime or produced a wrong answer early" and routing it to a diagnostic that reads the code.

Ranking and thresholding come apart. Runtime Error is the clearest case in which the model knows more than its label suggests. Its one-vs-rest ROC AUC is 0.831, against an F1 of 0.33 at the argmax (Figure 6); the macro one-vs-rest AUC is 0.955. The classifier orders runtime errors well above wrong answers—it has extracted real signal from the shared execution signature—but the two classes crowd together near the decision surface, so the single cut that argmax imposes cannot deliver both precision and coverage. This is what an equivalence class looks like from the model's side: the posterior is informative, the label is not.

The practical consequence is that abstention can be examined rather than merely recommended. Moving off the argmax to an explicit precision floor of 0.60 flags 38 submissions at precision 0.71 and recovers 21% of runtime errors, against 44 flagged at precision 0.66 and 22% recall under the argmax; a floor of 0.50 flags the same 44 submissions. The trades are marginal—raising the floor buys a little precision with coverage it does not repay—and we report them as an operating characteristic rather than a solution: they show that the RE posterior carries usable ordering information even though the RE verdict does not survive thresholding. Precision–recall curves for all five classes are released with the reproduction script.

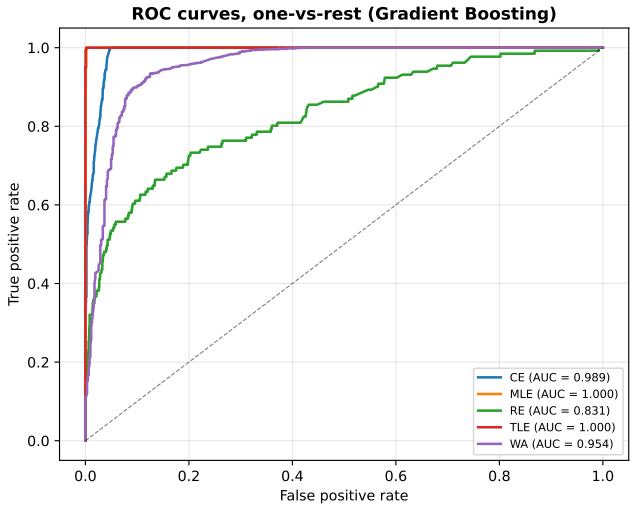


Fig. 6. One-vs-rest ROC curves, Gradient Boosting ($n = 2{,}672$). Runtime Error ranks well (AUC 0.831) despite an F1 of 0.33 at the argmax, because its errors are concentrated in the equivalence cell rather than spread across the score range. The gap between the two numbers is the space in which calibrated abstention operates.

5.4 LLM Error Analysis

The per-class behaviour of the two LLMs explains why their accuracy figures are a poor guide to what they have learned (Table 9).

DeepSeek-V3 is best understood as a threshold rule with no prior. It identifies every compilation error in the test partition—recall 1.00—by firing on submissions with very short execution times, and it recovers most TLE and MLE submissions (0.87 and 0.88). Because it applies the rule unconditionally, precision on CE falls to 0.321: 415 submissions whose true verdict is otherwise (385 wrong answers, 27 runtime errors and 3 TLEs) with short execution times— 137 reporting exactly 0 ms, the signature shown in the worked example, and the remaining 278 spanning 1–1000 ms (238 at $\leq$31 ms, the modal wrong-answer times)—are reported as compilation errors. Its aggregate accuracy, 76.65%, is indistinguishable from always answering Wrong Answer (76.61%), yet its balanced accuracy is 0.708, close to the supervised model's 0.804. The LLM applies the same platform encoding the tree ensemble found—it was shown one instance of the zero-time→CE mapping as a worked example and extended it to every short-time submission—; what it lacks is the calibrated trade-off inside

the equivalence cell—the ability to say "this cell is two thirds compilation errors, so I will report CE and accept the loss". Prompted with three worked examples and no prior over the cell, it can only apply the rule or not apply it.

The supervised model makes the opposite trade. It reports CE with precision 0.67 and recall 0.90, giving up 20 compilation errors to avoid 85 false ones, and spends the recovered capacity on WA. That difference, rather than any general superiority of gradient boosting over language models, is what the 15.1 percentage-point gap measures.

`Qwen2.5:3B` fails differently and more thoroughly. It spreads its predictions across the label set roughly in proportion to nothing in particular—CE 263, TLE 644, MLE 317, WA 715, RE 464, against a true WA share of 76.6%—so its balanced accuracy (0.348) is barely above chance for five classes, and 10.1% of its responses are unusable. We read this as a documented failure mode of small local models on numeric thresholding, not as evidence about the ordering of methods in general.

A note on the pilot. An earlier 207-submission pilot of DeepSeek-V3 (75.85%) used single-item prompting with no worked examples, a protocol superseded by the batched three-shot evaluation reported here; the two are not directly comparable. For completeness, the full 2,672-submission partition was also run under that superseded single-item protocol (DeepSeek-V3 78.29%, `Qwen2.5:3B` 11.34%), and the ordering of supervised models above both LLMs is unchanged ($78.29\% < 91.77\%$). The batched three-shot results are reported throughout; the episode illustrates why protocol details must be fixed before LLM comparisons are quoted.

What the worked examples do. The two superseded single-item runs and the three-shot evaluation share one test partition, which lets the examples' effect be measured rather than asserted. Without examples, DeepSeek-V3 answers Wrong Answer for 95.0% of submissions (2,539 of 2,672; prediction entropy 0.35 bits), recovering only 17.9% of compilation errors and 1.9% of time-limit verdicts—the platform encoding is present in the features but the model never applies it. With three worked examples the same model applies the encoding immediately: compilation-error recall rises to 1.00 and the prediction distribution spreads to 62.6% Wrong Answer (entropy 1.45 bits, 4.2× the no-example value), at the price of precision on the equivalence cell (0.32) and 1.6 pp of accuracy, because Wrong Answer is usually the right answer and the examples buy coverage of the execution-defined classes, not calibration of the prior inside the all-zero cell. The examples thus select *whether* the encoding is applied; they do not supply the calibrated prior that the cell would require. The two runs differ in call format as well as in example count (single-item versus eight per call), so the comparison fixes the mechanism's direction and size on this pair of runs rather than a controlled dose-response; a 15-example arm at fixed batching is the natural confirmation and is left to future work. (`experiment_code/fewshot_mechanism_analysis.py` reproduces every figure quoted here from the archived prediction files.)

5.5 Broader Implications for Computing Education Research

Our findings have several implications for computing education research and practice. First, the strong performance of metadata-only classification (91.8% accuracy without source code access) indicates that *submission logs alone* constitute a sufficiently rich signal for error pattern recognition, opening new possibilities for privacy-preserving educational analytics. Platforms that cannot access student source code due to privacy policies (e.g., proctored exams, IRB constraints) can still deploy effective verdict classification systems.

Second, the cost gap between traditional ML and frontier API models—four to six orders of magnitude per prediction at list prices—has practical significance for resource-constrained

Table 9. Per-class behaviour of the two LLMs on the shared test partition ($n = 2{,}672$; precision and recall computed over valid outputs only).

Metric	`Qwen2.5:3B`	DeepSeek-V3	Test partition
Valid outputs	2,403 (90.0%)	2,651 (99.2%)	—
Unusable (UNKNOWN)	269 (10.1%)	21 (0.8%)	—
CE precision / recall	0.21 / 0.31	0.32 / **1.00**	196 rows
TLE recall	0.39	0.87	258 rows
MLE recall	0.50	0.88	40 rows
WA recall	0.36	0.78	2,047 rows
RE recall	0.18	0.02	131 rows
RE F1	0.07	0.03	—
WA predicted share	29.8%	62.6%	76.6%
RE predicted share	19.3%	0.3%	4.9%
Balanced accuracy	0.348	0.708	—

Table 10. Method selection guidelines for different educational contexts. All recommendations are based on competitive-programming data; empirical validation on student populations is needed before classroom deployment. See Section 5.6 for population generalizability caveats.

Context	Method	Rationale
Post-submission verdict screening	Gradient Boosting	Fastest ensemble: about 5 μs per classification, $\approx\$10^{-7}$/1k compute
Automated grading dashboards	Gradient Boosting	Ensemble-tier accuracy (91.8%), best macro F1 (0.799), CV 92.4% ($\pm$0.33)
Personalized tutoring	Hybrid ML + LLM*	ML classification + LLM explanation
Privacy-sensitive cases	Random Forest	Local-only, no API calls, no third-party data transmission
Research / exploratory	Logistic Regression	Interpretable coefficients; establishes the interaction-structure floor (macro F1 0.49)
Low-resource classrooms	Random Forest	CPU-only, 0.75 s training, negligible cost

*The hybrid ML + LLM configuration was not empirically evaluated in this study; it is proposed as a direction for future work.

educational contexts. A semester-long deployment for 500 students generating approximately 500,000 submissions would cost approximately \$0.06 in Random Forest inference (about 11 ms per thousand on a single core) against roughly \$5 in DeepSeek-V3 API calls and \$47 at GPT-4o list prices; local Ollama inference has near-zero marginal cost (no API

fees, only electricity) but requires upfront hardware investment, so the trade-off is dominated by hardware availability rather than per-query fees. All three figures are small next to instructor time, which is what makes the latency and privacy differences, not price, the deciding factors. This cost structure is particularly relevant for MOOCs and competitive programming platforms serving millions of users.

5.5.1 Empirical Generalization Analysis. To assess generalizability beyond typical competitive programming distributions, we segmented the shared test partition into three problem-difficulty tiers—novice (rating <1,400), intermediate (1,400–1,999) and advanced (≥2,000)—and evaluated the Gradient Boosting model of Table 3 on each tier.

Accuracy declines with problem difficulty: 93.60% on intermediate problems (n=734, macro F1=0.766), 92.21% on novice problems (n=1,464, F1=0.794) and 87.55% on advanced problems (n=474, F1=0.811), with balanced accuracy of 0.777, 0.826 and 0.803 respectively. The decline is compositional rather than a failure of the classifier: advanced problems carry a more diverse verdict mix—TLE 17.5% and MLE 5.5% of the tier, against 3.3% and 0.2% for novice problems—and Runtime Error, the class the model handles worst overall (F1 0.33), is more frequent there (10.3% against 4.1%). RE F1 is non-monotonic across tiers (0.41 novice, 0.15 intermediate, 0.31 advanced); the intermediate tier contains only 22 runtime errors, so we read the pattern as composition and small-support noise rather than as difficulty-dependent degradation. WA recall stays above 0.94 in every tier.

This subpopulation analysis extends prior work on fine-grained error classification [4] by demonstrating that metadata-based ML generalizes across difficulty levels, with accuracy maintained at 87.6% even for the hardest problems, where the verdict mix is most diverse.

Population external validity: The skill-tier analysis demonstrates internal generalizability across competitive programming expertise levels, but the gap to CS1/CS2 student populations [48] remains open. Competitive programmers differ from novices in error cause and submission behavior (iterative learning under low time pressure versus competitive time pressure), and the two populations are measured differently: the largest novice corpus records only compiler-detected mistakes [4], so no published figure puts novice runtime-error rates beside ours. Validation on student datasets (*CS1QA* [28], Blackbox) is needed before drawing conclusions for novice education.

Theoretical CS1 projection: We cannot compute a submission-level weighted F1 for a CS1 population, because no published novice dataset records the verdict distribution we would need: the largest such corpus [4] contains only compiler-detected mistakes, and runtime and wrong-answer failures produce no diagnostic and therefore never enter it. What that corpus does show is the direction of the shift. Among the mistakes it can see, compile-time categories dominate—the single most frequent mistake alone (mismatched brackets) accounts for roughly a third of all occurrences—whereas compilation errors are 7.3% of our corpus. If a novice population moved even a modest share of its mass from WA toward CE, the weighted F1 would fall, because CE is the class the equivalence cell constrains most tightly (F1 0.77 against a 0.767 identifiability ceiling) while WA sits at 0.95. We therefore report the direction rather than a number, and we report it for the same reason we report the equivalence ceiling: a transfer claim is only as good as the assumption that per-class F1 is invariant across populations, and Section 5.3 gives reason to doubt it. In a population where compilation errors dominated, the all-zero cell would be composed differently, and CE F1 would move with the composition rather than staying at 0.77. Direct validation on student datasets (*CS1QA* [28], Blackbox) is required before any claim about CS1 populations.

Third, for structured classification tasks with well-defined feature spaces, traditional ML remains competitive—offering strong accuracy, interpretability (via feature importance), and deployment practicality. A *hybrid strategy* (ML for classification, LLM for explanation) leverages both paradigms.

Fourth, the gap between the 207-submission pilot (75.85%) and the full 2,672-submission evaluation (76.65% on valid outputs) for DeepSeek-V3 warns against quoting LLM numbers from small pilots whose prompting protocol differs from the final one; future work should report pilot and full-test results under an identical protocol with explicit sample representativeness discussion.

5.6 Limitations and Future Work

While our study provides a systematic comparison of ML and LLM methods for metadata-based verdict classification, several limitations should be acknowledged.

We tested two LLM configurations (DeepSeek-V3 and `Qwen2.5:3B`) under an identical three-shot protocol (Section 4.7); recent benchmarks [15] confirm substantial LLM family variation, and alternative model families (GPT-4o, Claude-3.5 Sonnet) would increase inference cost substantially—at standard list prices ($2.50/$10.00 and $3.00/$15.00 per million input/output tokens) the same eight-item protocol runs about 9× and 12× DeepSeek-V3 per 1,000 predictions. The three-example dose is itself a limitation: the worked examples decide whether the platform encoding is applied at all (Section 5.4), but the no-example arm differs from the reported arm in call format as well as example count, and a 15-example arm at fixed batching—the natural dose-response confirmation—was not run for lack of an API key at the time of writing.

Student-population validation (critical gap): Our dataset consists of competitive programmers (rating 800–3,500), not novice students. CS1/CS2 error distributions differ substantially: in the largest novice corpus, 47.6% of compilation events fail to compile [4], against 7.3% compilation-error verdicts in ours, and that corpus records compilation events only, so it offers no runtime-verdict figure to set beside our 4.9% RE share. Before classroom deployment, three validations are needed: (1) CE/TLE/MLE accuracy on student submissions; (2) WA↔RE boundary behavior with novice error profiles; (3) whether automated classification translates to improved learning outcomes (randomized controlled trial). We provide a theoretical CS1 projection (Section 5.5.1) and skill-tier analysis (Section 5.5.1) as partial evidence, but these do not substitute for direct student-population validation.

Linear Baseline Underperformance: Logistic Regression reaches 85.63% (Table 3), above the majority-class rule (76.61%) but 6.1 pp below the ensembles, and its Macro F1 collapses to 0.494 because it recovers 5 of 196 compilation errors. The deficiency is representational rather than statistical: a linear boundary cannot express the conjunction of zero-valued measurements that defines a compilation error. It is not repaired by tuning—the model converges in 91 iterations and the gap persists under submitter-disjoint evaluation (87.15%). We retain LR as an interpretable floor; it should not be deployed for verdict classification.

Paired-Test Independence Assumption: McNemar's test assumes independence across the 2,672 paired test instances, but 503 of 801 test users (63%) contribute multiple submissions clustered around a characteristic verdict (mean modal-verdict share 0.92; random baseline 0.20). We therefore report the McNemar p-values as a *lower bound* on uncertainty, not exact probabilities, and confirmed robustness through a fully user-disjoint replication (Section 4.9): Gradient Boosting retained 93.13% accuracy (92.15% CV) and the ML > LLM ranking held. A cluster-robust McNemar variant is recommended for

confirmatory replications seeking exact p-values. The small RE ($n = 131$) and MLE ($n = 40$) test subsets further limit per-class F1 precision.

Feature Set Limitations: We used six metadata fields and did not collect or evaluate source-code features (ASTs, token sequences, code embeddings). The claim that source-code analysis is *necessary* to resolve the WA↔RE boundary is therefore a hypothesis, though a better supported one than a bare failure would provide: the equivalence analysis of Section 5.3 shows that the two verdicts share execution signatures, and that more than half of the RE→WA errors occur inside cells shared with WA. What remains undemonstrated is that source features would suffice—we did not test them. Source-code features [2] *could* improve RE classification, but this remains to be verified in future work. More granular taxonomies (e.g., distinguishing segfault, division-by-zero, and assertion failure within RE) could enable more targeted feedback [4], but this is orthogonal to the metadata-separability question examined here.

Breadth Without Depth: The dataset is broad in programmers (4,005) but shallow per programmer (median two submissions). Any user-level quantity therefore rests on few observations per person: a WA-rate aggregate rests on a median of three earlier training submissions where any exist, adds no gain that survives correction, and is exactly zero on the submitter-disjoint split (Section 4.6). For the same reason the skill-tier analysis pools programmers by rating band rather than estimating individual trajectories. Conclusions about individual learners should not be drawn from this dataset; a course deployment that observes each student weekly would estimate these quantities far more precisely.

LLM Inference Cost: We used local Ollama (`Qwen2.5:3B`) at no API cost; future work should evaluate local LLM deployments for stable cost, performance, and privacy in educational settings.

Ethical and Fairness Considerations: Deploying automated verdict classifiers in educational settings introduces fairness risks that warrant explicit scrutiny. First, misclassified error types produce mis-targeted pedagogical feedback: a RE incorrectly labeled WA (or vice versa) can lead a learner to debug the wrong component or, worse, to believe a failing solution is correct. Second, this harm is not evenly distributed. Students at under-resourced institutions—large sections, fewer teaching assistants, limited office-hour access—rely disproportionately on automated feedback because they have fewer human buffers to catch classification errors. A false positive that a well-resourced peer would immediately overturn in a staffed lab may persist uncorrected for a under-resourced student across an entire assignment cycle. Third, the platform-encoding boundary (Section 3.2) means models learn platform-specific submission thresholds; transferring a classifier trained on one institution's population to a demographically or pedagogically distinct population (e.g., a school with a different error mix, more syntax errors, less competitive exposure) risks systematic, population-specific misclassification rather than random error. Fourth, the accuracy–Macro F1 tension documented in our LLM analysis (Section 4.7) generalizes: aggregate accuracy rewards the dominant WA class and can mask under-served minority errors (RE, MLE)—precisely the harder failures where under-resourced students most need accurate diagnosis. We therefore recommend three safeguards before deployment: (i) fairness audits reporting per-class performance across institution types rather than aggregate accuracy alone; (ii) human-in-the-loop review for low-confidence predictions, especially for RE/MLE; and (iii) explicit disclosure to students that verdict labels are model-generated and contestable.

Despite these limitations, our findings provide strong evidence that traditional ML methods remain highly competitive for metadata-based error classification, offering a cost-effective alternative to LLM-based approaches.

6 Threats to Validity

Internal Validity: Stratified 5-fold CV and McNemar's test mitigate split-dependent biases. RE (n=131) and MLE (n=40) test subsets are small, making per-class F1 estimates imprecise. CE/TLE/MLE predictions partially reflect platform threshold encoding; ablation (Table 6) shows 7.3pp drop but non-zero residual accuracy, confirming complementary signals.

External Validity: Our data comes from competitive programmers (median Codeforces rating below 1,400), and student error patterns differ (more CE, fewer complex RE). We evaluate cross-population generalizability via a skill-tier analysis (Section 5.5.1); the full gap analysis and validation agenda appear in Section 5.6.

Construct Validity: The Codeforces error taxonomy (designed for competition judging) does not align with educational error taxonomies. RE is heterogeneous (segfault, division by zero, etc.), and WA encompasses diverse root causes. A more nuanced taxonomy would provide greater educational utility.

Reproducibility Statement

All code, processed datasets, models, and the exact LLM prompts are publicly available at https://github.com/huwenbin123huwenbin/programming-error-classification. Random seeds are fixed (`random_state=42`), and `experiment_code/pipeline_authoritative.py` regenerates every number, table and figure in this paper from the released dataset in a single run. See Sections 3.4 and 3.5 for hyperparameters and evaluation protocol details. This research uses publicly available Codeforces data; the study was determined exempt from IRB review by Xijing University (exempt determination, reference 2024-AI-003; Section 7).

LLM Prompt Template (Three-Shot)

Both LLM configurations received the same three-shot prompt. Each request classified eight submissions at a time and carried three worked examples (one each for CE, TLE and MLE), so the protocol is three-shot rather than zero-shot; no fine-tuning was performed. The five metadata fields (`problem_rating`, `language`, `time_consumed_ms`, `memory_kb`, `passed_test_count`) were serialized one submission per line, with `language` normalized to a family (e.g., "C++23 (GCC 14-64, msys2)" → "C++"). Temperature was 0 for both models. DeepSeek-V3 additionally received the system message "*You are a competitive-programming error classifier. Respond with only a valid JSON object.*" and used grammar-constrained JSON output; `Qwen2.5:3B` used Ollama's JSON format constraint. The model returned a JSON object with key `verdicts` holding one label per item from the set {WA, TLE, RE, CE, MLE}; items whose position could not be parsed were counted as invalid outputs (DeepSeek-V3 21/2,672; `Qwen2.5:3B` 269/2,672). An earlier single-item zero-shot variant was superseded by this protocol (Section 5.4). The two instruction lines below are wrapped for display; the prompt sent to the models joined each into a single line.

```
Classify each competitive-programming submission error into exactly one of:
WA, TLE, RE, CE, MLE.
Example rating=1400 lang=C++ time_ms=0 mem_kb=0 passed=0 -> CE
```

```
Example rating=1800 lang=C++ time_ms=1987 mem_kb=1024 passed=0 -> TLE
Example rating=2000 lang=C++ time_ms=500 mem_kb=262144 passed=0 -> MLE
Classify each item. Return a JSON object with key 'verdicts' =
array of one token per item, in the same order.
[0] rating=1400 lang=C++ time_ms=15 mem_kb=0 passed=0
[1] rating=1000 lang=Kotlin time_ms=156 mem_kb=0 passed=0
[2] rating=1300 lang=C++ time_ms=15 mem_kb=100 passed=0
[3] rating=1300 lang=C++ time_ms=2000 mem_kb=6300 passed=1
[4] rating=1800 lang=C++ time_ms=0 mem_kb=0 passed=0
[5] rating=2200 lang=Java time_ms=0 mem_kb=0 passed=0
[6] rating=800 lang=C++ time_ms=62 mem_kb=0 passed=1
[7] rating=1200 lang=C++ time_ms=31 mem_kb=0 passed=1
Verdicts:
```

Ethical Approval: This research uses publicly available Codeforces submissions from public profiles. No private student data was collected and no human participants were involved. The research was reviewed and determined to be *exempt* from IRB review by the Institutional Review Board of Xijing University (exempt determination, reference 2024-AI-003). Because the determination is exempt rather than approved, no approval number is issued; the reference number identifies the determination only.

7 Conclusion

This study asks how much verdict information a submission log carries. The answer is not a single accuracy figure but a decomposition. Of the 13,360 submissions, 18.5% fall in the three platform-measured verdict classes—Time Limit Exceeded (F1 0.99) and Memory Limit Exceeded (0.95) are fully recoverable because the judge's own thresholds define them, while Compilation Error (0.77) is recoverable only up to the point where its zero-millisecond signature coincides with an early crash or a first-test wrong answer. That coincidence—11.3% of the corpus lying in one signature cell that spans three verdicts—is what limits metadata-only classification, and the Wrong Answer↔Runtime Error boundary is the same phenomenon with a different pair of verdicts. A gradient-boosted model over six metadata fields reaches 91.77% (balanced accuracy 0.80), leads the strongest prompted LLM by 15.1 pp at four to five orders of magnitude lower per-prediction inference cost, and is unchanged under submitter-disjoint evaluation (93.13%), which shows that the estimate does not depend on seeing the same person twice. Execution time is the dominant predictor (7.3 pp drop, 81.5% of Gini importance), and a submitter-history aggregate—the standard first feature in this literature—adds nothing reliable: a nominal 0.67 pp that does not survive correction, and exactly zero when submitters are disjoint.

Future directions. Three next steps follow from the equivalence account rather than from the model comparison. First, **audit the cells elsewhere**: computing the same signature analysis on LeetCode or AtCoder logs would show whether the all-zero and first-test-failure cells are a general property of judge architectures or an artefact of how Codeforces reports. Second, **collect the two missing measurements**: the compiler diagnostic and the index of the first failing test are held by every major judge but not published. Recovering them for a sample would let one measure directly how much of each equivalence class splits—a sharper experiment than adding source code wholesale, and one that yields a cost curve for platform designers. Third, **classroom validation**: the same pipeline run on a course deployment,

where per-student histories are long enough for user-level features to be estimated rather than guessed.

Practically, the boundary tells platform developers where to draw the line between automated and code-level feedback. The execution-encoded verdicts are exactly those for which immediate, specific feedback is both cheap and actionable: a compilation error warrants syntax guidance, a time-limit verdict warrants complexity analysis. At the Wrong Answer↔Runtime Error boundary the system should abstain rather than guess, report the ambiguity, and route the submission to a diagnostic that reads the code; calibrated abstention is itself pedagogically useful, because it tells the learner which question to ask. A hybrid deployment—ML for fast, cheap classification, LLMs invoked only at the boundary for explanation—follows from this division of labour, though we emphasise that the hybrid configuration was not evaluated here.

Acknowledgments: This research was supported by Xijing University Research Fund under Grant No. XJ-2025-001.

Conflicts of Interest: The authors declare no financial or non-financial conflicts of interest. This work used the DeepSeek-V3 API (paid inference) and the open-source `Qwen2.5:3B` model via Ollama; no commercial relationship exists with either provider. The funding body (Xijing University) had no role in the design, execution, analysis, or reporting of this study.

Preregistration: This study was not preregistered. All analyses reported were specified after data collection; we note this as a limitation and recommend preregistration for future confirmatory replications.

Data Ethics and Privacy

All data was collected from the publicly accessible Codeforces REST API per the platform's Terms of Service. Usernames were anonymized. This study was determined to be exempt from IRB review under standard guidelines for secondary analysis of publicly available data (Xijing University IRB; exempt determination, reference 2024-AI-003; no approval number is issued for exempt determinations). Raw payloads are retained locally and not redistributed per platform Terms of Service.

Data and Code Availability

Code and processed datasets are available at https://github.com/huwenbin123huwenbin/programming-error-classification.

References

[1] A. Ahadi, R. Lister, H. Haapala, and A. Vihavainen. 2015. Exploring Machine Learning Methods to Automatically Identify Students in Need of Assistance. In *Proceedings of the 11th Annual International Conference on International Computing Education Research*. ACM, 121–130. doi:10.1145/2787622.2787717

[2] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to represent programs with graphs. In *International Conference on Learning Representations (ICLR)*. arXiv:1711.00740

[3] A. Alnahhas and N. Mourtada. 2020. Predicting the Performance of Contestants in Competitive Programming Using Machine Learning Techniques. *Olympiads in Informatics* 14 (2020), 3–20. doi:10.15388/ioi.2020.01

[4] Amjad Altadmri and Neil C. C. Brown. 2015. 37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. ACM, 522–527. doi:10.1145/2676723.2677258

[5] Ryan S. J. d. Baker and Kalina Yacef. 2009. The State of Educational Data Mining in 2009: A Review and Future Visions. *Journal of Educational Data Mining* 1, 1 (2009), 3–17. doi:10.5281/zenodo.3554657

[6] Albert Bandura. 1997. *Self-efficacy: The Exercise of Control*. Freeman.
[7] H. J. Barbosa Rocha, P. Cabral de A. Tedesco, and E. de Barros Costa. 2023. On the Use of Feedback in Learning Computer Programming by Novices: A Systematic Literature Mapping. *Informatics in Education* 22, 2 (2023), 209–232. doi:10.15388/infedu.2023.09
[8] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. In *Proceedings of the ACM on Programming Languages*, Vol. 7. ACM, 85:1–85:29. doi:10.1145/3586030
[9] Brett A. Becker, Paul Denny, James Finnie-Ansley, et al. 2023. Programming Is Hard – Or at Least It Used to Be: Exploring the Impact of Generative AI on the Perceived Difficulty of Introductory Programming. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE '23)*. ACM, 500–506. doi:10.1145/3545945.3569759
[10] J. Bennedsen and M. E. Caspersen. 2007. Failure rates in introductory programming. *ACM SIGCSE Bulletin* 39, 2 (2007), 32–36. doi:10.1145/1272848.1272879
[11] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324
[12] Neil C. C. Brown, Michael Kölling, Davin McCall, and Ian Utting. 2014. Blackbox: A Large Scale Repository of Novice Programmers' Activity. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education (SIGCSE '14)*. ACM, New York, NY, 223–228. doi:10.1145/2538862.2538924
[13] Teresa Busjahn, Roman Bednarik, and Andrew Begel. 2015. Eye Movements in Code Reading: Relaxing the Linear Order. In *2015 IEEE International Conference on Program Comprehension*. IEEE, 255–264. doi:10.1109/icpc.2015.36
[14] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv preprint arXiv:2412.19437. doi:10.48550/arXiv.2412.19437
[15] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A. Becker, and Brent N. Reeves. 2024. Prompt Problems: A New Programming Exercise for the Generative AI Era. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. ACM, 296–302. doi:10.1145/3626252.3630909
[16] T. Effenberger and R. Pelánek. 2021. Validity and Reliability of Student Models for Problem-Solving Activities. In *Proceedings of the 11th International Conference on Learning Analytics and Knowledge*. ACM, 1–11. doi:10.1145/3448139.3448140
[17] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Peng, Bing Qin, Ting Liu, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
[18] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proceedings of the 24th Australasian Computing Education Conference*. ACM, 10–19. doi:10.1145/3511861.3511863
[19] Jerome H Friedman. 2001. Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29, 5 (2001), 1189–1232. doi:10.1214/aos/1013203451
[20] Jamie Gorson and Eleanor O'Rourke. 2020. Why do CS1 Students Think They're Bad at Programming?: Investigating Self-efficacy and Self-assessments at Three Universities. In *Proceedings of the 2020 ACM Conference on International Computing Education Research*. ACM, 170–181. doi:10.1145/3372782.3406273
[21] John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487
[22] David W. Hosmer, Stanley Lemeshow, and Rodney X. Sturdivant. 2013. *Applied Logistic Regression* (3 ed.). John Wiley & Sons. doi:10.1002/9781118548387
[23] Matthew C. Jadud. 2006. Methods and tools for exploring novice compilation behaviour. In *Proceedings of the Second International Workshop on Computing Education Research*. ACM, 73–84. doi:10.1145/1151588.1151600
[24] Sandeep M. Jayaprakash, Erik W. Moody, E. J. M. Lauria, James R. Regan, and Joshua D. Baron. 2014. Early Alert of Academically At-Risk Students: An Open Source Analytics Initiative. *Journal of Learning Analytics* 1, 1 (2014), 6–47. doi:10.18608/jla.2014.11.3
[25] H. Keuning, J. Jeuring, and B. Heeren. 2018. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Transactions on Computing Education* 19, 1 (2018), 3:1–3:43. doi:10.1145/3231711

[26] Paul A. Kirschner, John Sweller, Femke Kirschner, and Jacobo Zambrano R. 2018. From Cognitive Load Theory to Collaborative Cognitive Load Theory. *International Journal of Computer-Supported Collaborative Learning* 13 (2018), 213–233. doi:10.1007/s11412-018-9277-y

[27] C. Koutcheme, N. Dainese, S. Sarsa, A. Hellas, J. Leinonen, and P. Denny. 2024. Open Source Language Models Can Provide Feedback: Evaluating LLMs' Ability to Help Students Using GPT-4-As-A-Judge. In *Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education V. 1*. ACM, 52–58. doi:10.1145/3649217.3653612

[28] C. Lee, Y. Seonwoo, and S. Oh. 2022. CS1QA: A Dataset for Assisting Code-Based Question Answering in an Introductory Programming Course. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2026–2040. doi:10.18653/v1/2022.naacl-main.148

[29] J. Leinonen, A. Hellas, S. Sarsa, B. Reeves, P. Denny, J. Prather, and B. A. Becker. 2023. Using Large Language Models to Enhance Programming Error Messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023)*. ACM, 563–569. doi:10.1145/3545945.3569770

[30] Y. Liu, S. Gautam, J. Ma, and H. Lakkaraju. 2024. Confronting LLMs with Traditional ML: Rethinking the Fairness of Large Language Models in Tabular Classifications. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*. arXiv:2310.14607. doi:10.18653/v1/2024.naacl-long.198

[31] A. Luxton-Reilly, Simon, I. Albluwi, B. A. Becker, M. Giannakos, A. N. Kumar, L. Ott, J. Paterson, M. J. Scott, J. Sheard, and C. Szabo. 2018. Introductory programming: a systematic literature review. In *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*. ACM, 55–106. doi:10.1145/3293881.3295779

[32] R. McCauley, S. Fitzgerald, G. Lewandowski, L. Murphy, B. Simon, L. Thomas, and C. Zander. 2008. Debugging: a review of the literature from an educational perspective. *Computer Science Education* 18, 2 (2008), 67–92. doi:10.1080/08993400802114581

[33] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996

[34] M. Mirzayanov, O. Pavlova, I. Mavrin, A. Melnikov, and A. Plotnikov. 2020. Codeforces as an Educational Platform for Learning Programming in Digitalization. *Olympiads in Informatics* (2020), 133–142. doi:10.15388/ioi.2020.10

[35] Martin Monperrus. 2018. Automatic Software Repair. *Comput. Surveys* 51, 1 (2018), 1–24. doi:10.1145/3105906

[36] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

[37] A. Petersen, J. Spacco, and A. Vihavainen. 2015. An Exploration of Error Quotient in Multiple Contexts. In *Proceedings of the 15th Koli Calling International Conference on Computing Education Research (Koli Calling 2015)*. ACM, 77–86. doi:10.1145/2828959.2828966

[38] Chris Piech, Jonathan Spencer, and Jonathan Huang. 2015. Deep Knowledge Tracing. *arXiv preprint arXiv:1506.05908* (2015). arXiv:1506.05908 [cs.AI] doi:10.48550/arXiv.1506.05908

[39] J. Prather, J. Leinonen, N. Kiesler, J. Gorson Benario, S. Lau, S. MacNeil, N. Norouzi, S. Opel, V. Pettit, L. Porter, B. N. Reeves, J. Savelka, D. H. Smith, S. Strickroth, and D. Zingaro. 2024. How Instructors Incorporate Generative AI into Teaching Computing. In *Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education V. 2*. ACM, 771–772. doi:10.1145/3649405.3659534

[40] N. Raihan, M. Latif Siddiq, J. C. S. Santos, and M. Zampieri. 2025. Large Language Models in Computer Science Education: A Systematic Literature Review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education*. ACM, 937–943. doi:10.1145/3641554.3701863

[41] C. Romero and S. Ventura. 2013. Data mining in education. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 3, 1 (2013), 12–27. doi:10.1002/widm.1075

[42] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1*. ACM, 27–43. doi:10.1145/3501385.3543957

[43] J. C. Spohrer and E. Soloway. 1986. Novice mistakes: are the folk wisdoms correct? *Commun. ACM* 29, 7 (1986), 624–632. doi:10.1145/6138.6145

[44] John Sweller. 2011. Cognitive Load Theory. In *Psychology of Learning and Motivation*, Jose P. Mestre and Brian H. Ross (Eds.). Vol. 55. Academic Press, 37–76. doi:10.1016/B978-0-12-387691-1.00002-8

[45] Katrien Verbert, Erik Duval, and Joris Klerkx. 2013. Learning Analytics Dashboard Applications. *American Behavioral Scientist* 57, 10 (2013), 1500–1509. doi:10.1177/0002764213479363

[46] C. Watson and F. W. Li. 2014. Failure rates in introductory programming revisited. In *Proceedings of the 2014 Conference on Innovation & Technology in Computer Science Education*. ACM, 39–44. doi:10.1145/2591708.2591749

[47] Leon E. Winslow. 1996. Programming Pedagogy: A Psychological Overview. *ACM SIGCSE Bulletin* 28, 3 (1996), 17–22. doi:10.1145/234867.234872

[48] Aman Yadav, Chris Mayfield, Sukanya Kannan Moudgalya, Clif Kussmaul, and Helen H. Hu. 2021. Collaborative Learning, Self-Efficacy, and Student Performance in CS1 POGIL. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*. ACM, 775–781. doi:10.1145/3408877.3432373

[49] Jialu Zhang, De Li, John Charles Kolesar, Hanyuan Shi, and Ruzica Piskac. 2022. Automated Feedback Generation for Competition-Level Code. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1–13. doi:10.1145/3551349.3560425